



ΕΘΝΙΚΟ ΜΕΤΣΟΒΙΟ ΠΟΛΥΤΕΧΝΕΙΟ
ΣΧΟΛΗ ΗΛΕΚΤΡΟΛΟΓΩΝ ΜΗΧΑΝΙΚΩΝ ΚΑΙ ΜΗΧΑΝΙΚΩΝ ΥΠΟΛΟΓΙΣΤΩΝ
ΤΟΜΕΑΣ ΤΕΧΝΟΛΟΓΙΑΣ ΠΛΗΡΟΦΟΡΙΚΗΣ ΚΑΙ ΥΠΟΛΟΓΙΣΤΩΝ

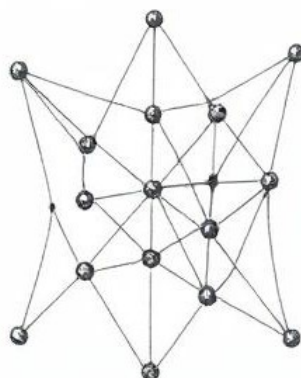
Βέλτιστη Offline Πλήρως Δυναμική 2-Ακμική Συνεκτικότητα

Μελέτη και υλοποίηση

ΔΙΠΛΩΜΑΤΙΚΗ ΕΡΓΑΣΙΑ

ΤΟΥ

ΙΩΑΝΝΗ ΡΙΖΑ



Επιβλέπων: Δημήτριος Φωτάκης
Καθηγητής, Ε.Μ.Π.

Συνεπιβλέπων: Λουκάς Γεωργιάδης
Καθηγητής, Παν. Ιωαννίνων

Αθήνα, Ιούλιος 2025



Βέλτιστη Offline Πλήρως Δυναμική 2-Ακμική Συνεκτικότητα

Μελέτη και υλοποίηση

ΔΙΠΛΩΜΑΤΙΚΗ ΕΡΓΑΣΙΑ

του

ΙΩΑΝΝΗ ΡΙΖΑ

Επιβλέπων: Δημήτριος Φωτάκης
Καθηγητής, Ε.Μ.Π.

Συνεπιβλέπων: Λουκάς Γεωργιάδης
Καθηγητής, Παν. Ιωαννίνων

Εγκρίθηκε από την τριμελή εξεταστική επιτροπή στις 14 Ιουλίου 2025.

(Υπογραφή)

(Υπογραφή)

(Υπογραφή)

.....
Δημήτριος Φωτάκης
Καθηγητής, Ε.Μ.Π.

.....
Λουκάς Γεωργιάδης
Καθηγητής, Παν. Ιωαννίνων

.....
Αριστείδης Παγουρτζής
Καθηγητής, Ε.Μ.Π.



Copyright © – All rights reserved. Με την επιφύλαξη παντός δικαιώματος.

Ιωάννης Ριζάς, 2025.

Απαγορεύεται η αντιγραφή, αποθήκευση και διανομή της παρούσας εργασίας, εξ ολοκλήρου ή τμήματος αυτής, για εμπορικό σκοπό. Επιτρέπεται η ανατύπωση, αποθήκευση και διανομή για σκοπό μη κερδοσκοπικό, εκπαιδευτικής ή ερευνητικής φύσης, υπό την προϋπόθεση να αναφέρεται η πηγή προέλευσης και να διατηρείται το παρόν μήνυμα.

Το περιεχόμενο αυτής της εργασίας δεν απηχεί απαραίτητα τις απόψεις του Τμήματος, του Επιβλέποντα, ή της επιτροπής που την ενέκρινε.

ΔΗΛΩΣΗ ΜΗ ΛΟΓΟΚΛΟΠΗΣ ΚΑΙ ΑΝΑΛΗΨΗΣ ΠΡΟΣΩΠΙΚΗΣ ΕΥΘΥΝΗΣ

Με πλήρη επίγνωση των συνεπειών του νόμου περί πνευματικών δικαιωμάτων, δηλώνω ενυπογράφως ότι είμαι αποκλειστικός συγγραφέας της παρούσας Πτυχιακής Εργασίας, για την ολοκλήρωση της οποίας κάθε βοήθεια είναι πλήρως αναγνωρισμένη και αναφέρεται λεπτομερώς στην εργασία αυτή. Έχω αναφέρει πλήρως και με σαφείς αναφορές, όλες τις πηγές χρήσης δεδομένων, απόψεων, θέσεων και προτάσεων, ιδεών και λεκτικών αναφορών, είτε κατά κυριολεξία είτε βάσει επιστημονικής παράφρασης. Αναλαμβάνω την προσωπική και ατομική ευθύνη ότι σε περίπτωση αποτυχίας στην υλοποίηση των ανωτέρω δηλωθέντων στοιχείων, είμαι υπόλογος έναντι λογοκλοπής, γεγονός που σημαίνει αποτυχία στην Πτυχιακή μου Εργασία και κατά συνέπεια αποτυχία απόκτησης του Τίτλου Σπουδών, πέραν των λοιπών συνεπειών του νόμου περί πνευματικών δικαιωμάτων. Δηλώνω, συνεπώς, ότι αυτή η Πτυχιακή Εργασία προετοιμάστηκε και ολοκληρώθηκε από εμένα προσωπικά και αποκλειστικά και ότι, αναλαμβάνω πλήρως όλες τις συνέπειες του νόμου στην περίπτωση κατά την οποία αποδειχθεί, διαχρονικά, ότι η εργασία αυτή ή τμήμα της δεν μου ανήκει διότι είναι προϊόν λογοκλοπής άλλης πνευματικής ιδιοκτησίας.

(Υπογραφή)

.....

Ιωάννης Ριζάς

14 Ιουλίου 2025

Περίληψη

Έστω ένα ακατεύθυντο, αβαρές κι απλό γράφημα $G = (V, E)$, όπου V είναι το σύνολο των κορυφών και E το σύνολο των ακμών. Επίσης, δίνεται μία ακολουθία λειτουργιών (operations) τριών διαφορετικών τύπων: εισαγωγές ακμών, διαγραφές ακμών και ερωτήματα συνεκτικότητας (queries) σχετικά με το αν ένα ζεύγος κορυφών είναι 2-ακμικά συνδεδεμένο (2-edge connected). Δεδομένης μιας ακολουθίας τέτοιων λειτουργιών που θα εφαρμοστούν πάνω στο αρχικό γράφημα G και είναι όλες γνωστές εκ των προτέρων (offline), χρειάζεται να απαντηθούν όλα τα ερωτήματα συνεκτικότητας βέλτιστα.

Στόχος είναι η υλοποίηση ενός ασυμπτωτικά βέλτιστου αλγορίθμου, ο οποίος απαντά όλα τα ερωτήματα συνεκτικότητας σε συνολικό χρόνο $O(t \log n)$, όπου t είναι το μήκος της ακολουθίας λειτουργιών και n το πλήθος κορυφών του γραφήματος. Ο αλγόριθμος εφαρμόζει μια αναδρομική στρατηγική διαίρει-και-βασίλευε πάνω στις λειτουργίες, επιτρέποντας τη συμπίκνωση της πληροφορίας του γραφήματος σε κάθε αναδρομική κλήση. Όταν το μέγεθος των λειτουργιών γίνει τετριμμένο, τα ερωτήματα συνεκτικότητας πλέον μπορούν να απαντηθούν σε σταθερό χρόνο.

Λέξεις Κλειδιά

γραφήματα, δυναμικοί αλγόριθμοι, 2-ακμική συνεκτικότητα, συνεκτικότητα ακμών, δυναμική συνεκτικότητα, πλήρως δυναμικά γραφήματα, διαίρει και βασίλευε, offline, αραιοποίηση γραφημάτων, τεχνικές αραιοποίησης

Abstract

Consider an undirected, unweighted, and simple graph denoted as $G = (V, E)$, where V represents the set of vertices and E represents the set of edges. We are given a sequence of operations of three different types: edge insertions, edge deletions, and connectivity queries that ask whether a given pair of vertices are 2-edge connected. Given that all operations are known in advance (offline), our objective is to answer all connectivity queries optimally.

We aim to design and implement an asymptotically optimal algorithm, capable of processing the entire sequence of operations in $O(t \log n)$ time, where t is the number of operations and n is the number of vertices in the original graph. The proposed algorithm follows a recursive divide-and-conquer approach over the operations sequence, allowing for the progressive compression and sparsification of the graph structure at each level of recursion. By reducing the graph size and eliminating irrelevant data, the algorithm guarantees that queries on small instances can be answered in constant time. This approach ensures both theoretical efficiency and practical scalability for offline fully dynamic 2-edge connectivity testing.

Keywords

graphs, dynamic algorithms, 2-edge connectivity, dynamic connectivity, fully dynamic graphs, divide and conquer, offline, graph sparsification, sparsification techniques

στη Λένα, το Λεωνίδα, το Δημήτρη και τη Νίτσα

Περιεχόμενα

Περίληψη	1
Abstract	2
Πρόλογος	9
1 Εισαγωγή	10
1.1 Ιστορική Εξέλιξη της Δυναμικής Συνεκτικότητας	10
1.2 Αντικείμενο της Εργασίας	11
1.3 Οργάνωση της Εργασίας	11
I Θεωρητικό Μέρος	13
2 Θεωρητικό υπόβαθρο	14
2.1 Δυναμικά Γραφήματα	14
2.1.1 Στατικά και Δυναμικά Γραφήματα	14
2.1.2 Λειτουργίες	14
2.1.3 Πλήρως και Μερικώς Δυναμικά Προβλήματα	15
2.1.4 Online και Offline	15
2.2 Συνεκτικότητα	16
2.2.1 Ακμική και Κομβική Συνεκτικότητα	16
2.2.2 Ερωτήματα Συνεκτικότητας	16
2.3 Υπολογισμός Δισυνεκτικών Δυνιστωσών	17
2.3.1 Ακμική Δισυνεκτικότητα	17
2.3.2 Κομβική Δισυνεκτικότητα	18
3 Offline 2-Ακμική Συνεκτικότητα	19
3.1 Ορισμοί	19
3.1.1 Περιγραφή του Προβλήματος	19
3.1.2 Απλοποίηση Δεδομένων Εισόδου	20
3.1.3 Θεωρητικό Μοντέλο	20
3.1.4 Ανάλυση Χρονικής Πολυπλοκότητας	20
4 Βασικές Υπορουτίνες	22
4.1 Επεξεργασία Ακολουθίας Λειτουργιών	22
4.1.1 Ενεργές Κορυφές	22

4.1.2	Διάρκεια Ζωής μίας Ακμής	22
4.1.3	Μόνιμες Ακμές	23
4.2	Επεξεργασία Γραφήματος (Ισοδύναμο Γράφημα)	25
4.2.1	Αντιπρόσωποι	25
4.2.2	Ακμικές Δισυνεκτικές Συνιστώσες (Δεντρικό Γράφημα)	26
4.2.3	Ακμικές Δισυνεκτικές Συνιστώσες με Ενεργές Κορυφές	28
4.2.4	“Κλάδεμα” Δεντρικού Γραφήματος	28
4.2.5	Ανανέωση του Πίνακα Αντιπροσώπων	32
4.2.6	Ισοδύναμο Γράφημα	32
5	Ανάλυση και Σχεδίαση	35
5.1	Βέλτιστο Αλγοριθμικό Μοντέλο	35
5.1.1	Δεδομένα Εισόδου	35
5.1.2	Αρχιτεκτονική Αλγορίθμου	35
5.2	Απλός Αλγόριθμος Αναφοράς	38
II	Πειραματικό Μέρος	39
6	Πειράματα & Αποτελέσματα	40
6.1	Παραγωγή Εισόδου	40
6.2	Διαγράμματα Αποτελεσμάτων	41
6.2.1	Περιπτώσεις Μικρής Κλίμακας ($N \in \{10, 40, 100\}$)	41
6.2.2	Περιπτώσεις Μεσαίας Κλίμακας ($N \in \{400, 1000, 3000\}$)	42
6.2.3	Περιπτώσεις Μεγάλης Κλίμακας ($N \in \{6000, 10000\}$)	43
6.3	Συμπεράσματα	45
III	Επίλογος	46
7	Επίλογος	47
7.1	Μελλοντικές Επεκτάσεις	47
	Παραρτήματα	49
A'	Ορισμοί, Λήματα & Θεωρήματα	50
A'.1	Γενικά	50
A'.2	Ακμική Δισυνεκτικότητα	51
B'	Θεμελιώδες Κάτω Φράγμα για Δυναμική Συνεκτικότητα	52
Γ'	Πολυπλοκότητα Χρόνου σε Σχέση με το Μέγεθος Εισόδου	53
Δ'	Κατηγοριοποίηση Βιβλιογραφικών Αναφορών	55
	Βιβλιογραφία	58

Συντομογραφίες - Αρκτικόλεξα - Ακρωνύμια	59
Απόδοση ξενόγλωσσων όρων	60

Κατάλογος Σχημάτων

2.1	Παράδειγμα ενός απλού μη-κατευθυνόμενου γραφήματος	17
2.2	Αναπαράσταση του Παραγόμενου Δεντρικού Γραφήματος	17
2.3	Παράδειγμα μη κομβικού δισυνεκτικού γραφήματος. Με κόκκινο φαίνονται οι αρ- θρώσεις του	18
2.4	Παράδειγμα κομβικού δισυνεκτικού γραφήματος	18
3.1	Παράδειγμα εισόδου με: 1.αρχικό γράφημα και, 2.ακολουθία λειτουργιών.	19
3.2	Παραλλαγή εισόδου: Εκτεταμένη ακολουθία λειτουργιών από το Σχήμα 3.1.	20
4.1	Παράδειγμα διάρκειας ζωής ακμών	23
4.2	Παράδειγμα αρχικού γραφήματος. Με πράσινο φαίνονται οι ενεργές κορυφές του	28
4.3	Εύρεση των γεφυρών του γραφήματος	28
4.4	Παραγόμενο δεντρικό ισοδύναμο γράφημα από το αρχικό	28
4.5	Παράδειγμα αφαίρεσης ανενεργής κορυφής βαθμού 0 και 1	29
4.6	Παράδειγμα αφαίρεσης ανενεργής κορυφής βαθμού 2	29
4.7	Αναπαράσταση των σχέσεων των διαδικασιών μεταξύ των επεξεργασιών ενός γραφήματος	32
4.8	Παράδειγμα από το αρχικό στο ισοδύναμο γράφημα	32
4.9	Πίνακας Αντιπροσώπων Κορυφών Κλαδεμένου/Ισοδύναμου Γραφήματος	33
6.1	Γράφημα 1: $G(V = 10, E = 19)$	41
6.2	Γράφημα 2: $G(V = 40, E = 71)$	42
6.3	Γράφημα 3: $G(V = 100, E = 175)$	42
6.4	Γράφημα 4: $G(V = 400, E = 811)$	43
6.5	Γράφημα 5: $G(V = 1000, E = 1479)$	43
6.6	Γράφημα 6: $G(V = 3000, E = 3034)$	44
6.7	Γράφημα 7: $G(V = 6000, E = 9819)$	44
6.8	Γράφημα 8: $G(V = 10000, E = 10482)$	44

Κατάλογος Πινάκων

4.1	Ενεργές κορυφές: {1,2,3,9}	22
4.2	Παράδειγμα αρχικής ακολουθίας λειτουργιών	24
4.3	Επαυξημένη τελική ακολουθία λειτουργιών	24
Γ.1	Σχέση πολυπλοκότητας χρόνου με το μέγεθος εισόδου [1]	53

Πρόλογος

Η παρούσα διπλωματική εργασία εκπονήθηκε στο πλαίσιο των σπουδών στο Τμήμα Ηλεκτρολόγων Μηχανικών και Μηχανικών Υπολογιστών του Εθνικού Μετσόβιου Πολυτεχνείου, με αντικείμενο την ερευνητική ενασχόληση στη περιοχή των δυναμικών γραφημάτων και αλγορίθμων σε συνεργασία με το Πανεπιστήμιο Ιωαννίνων.

Συγκεκριμένα, η διπλωματική εργασία αφορά τη μελέτη και υλοποίηση ενός αποδοτικού αλγορίθμου για το offline πρόβλημα της δυσυνεκτικότητας (ή 2-ακμικής συνεκτικότητας), με στόχο την πειραματική επιβεβαίωση της θεωρητικής ανάλυσης και της πειραματικής αξιολόγησής της αποδοτικότητάς του, σε σύγκριση με απλοϊκές μεθόδους. Η υλοποίησή έγινε σε γλώσσα C++ αποτελώντας και το κύριο και πιο απαιτητικό μέρος του εγχειρήματος αυτής της εργασίας.

Η θεωρητική βάση της υλοποίησης βασίστηκε στην εργασία των R. Peng, B. Sandlund & D. Sleator[2], η οποία πραγματεύεται βέλτιστους offline αλγορίθμους για δυναμική δυσυνεκτικότητα και τρισυνεκτικότητα ακμών και κορυφών σε ακατεύθυντα γραφήματα, καθώς και για τετραπλή ακμική και κομβική συνεκτικότητα. Η μελέτη αυτή αποτελεί σημείο αναφοράς για αμιγώς offline δυναμικά προβλήματα συνεκτικότητας και προσφέρει αποδοτικές λύσεις με θεμελιώδη θεωρητική τεκμηρίωση.

Η υλοποίηση του αλγορίθμου σε γλώσσα C++ για το βέλτιστο, αμιγώς offline, πλήρως δυναμικό, 2-ακμικό, πρόβλημα συνεκτικότητας είναι διαθέσιμη στο GitHub:

<https://github.com/dmiw189/offline-2-edge-connectivity>

1.1 Ιστορική Εξέλιξη της Δυναμικής Συνεκτικότητας

Η σημασία των δυναμικών προβλημάτων συνεκτικότητας έγκειται στην ικανότητά τους να εντοπίζουν κρίσιμες συνδέσεις και να μετρούν την *αυθεκτικότητα* του γραφήματος έναντι αλλαγών. Όταν η δομή ενός γραφήματος μεταβάλλεται δυναμικά — μέσω προσθήκης ή διαγραφής ακμών και κορυφών — το πρόβλημα της συνεκτικότητας γίνεται πολύ πιο απαιτητικό από τη στατική του εκδοχή και απαιτεί εξειδικευμένες αλγοριθμικές τεχνικές.

Οι πρώτες σημαντικές εξελίξεις εντοπίζονται στα μέσα της δεκαετίας του 1990. Το 1994, παρουσιάστηκε αλγόριθμος που επιλύει το πρόβλημα του *Ελάχιστου Συνδεδειμένου Δέντρου* (minimum spanning tree) σε offline περιβάλλον με χρόνο $O(\log n)$ ανά λειτουργία [3]. Το 1996 ακολούθησε η πρώτη σημαντική πρόοδος σε γενικά προβλήματα δυναμικής συνεκτικότητας, με αλγορίθμους για απλή συνεκτικότητα και δισυνεκτικότητα -τόσο ακμική όσο και κομβική- που επιτυγχάνουν χρόνο $\tilde{O}(\sqrt{n})$ ανά πράξη [4, 5, 6, 7, 8].

Το 1997 έγιναν οι πρώτες προσπάθειες για αντιμετώπιση πιο σύνθετων μορφών συνεκτικότητας, όπως η 3-ακμική και η 3-κομβική online συνεκτικότητα, με πολυπλοκότητες $O(n^{2/3})$ και $O(n)$ αντίστοιχα [9].

Το 2001, παρουσιάστηκαν βελτιώσεις που κατέβασαν σημαντικά τις πολυπλοκότητες: $O(\log^2 n)$ για απλή συνεκτικότητα, $O(\log^4 n)$ για ακμική δισυνεκτικότητα και $O(\log^5 n)$ για κομβική δισυνεκτικότητα [10].

Το 2004, οι Patrascu & Demaine απέδειξαν ότι κάθε δυναμικό πρόβλημα συνεκτικότητας — τόσο στην online όσο και στην offline εκδοχή του — υπόκειται σε θεωρητικό κάτω φράγμα χρόνου $\Omega(\log n)$ ανά πράξη [11, 12]. Αυτό έθεσε ένα ορόσημο που καθοδηγεί μέχρι και σήμερα την έρευνα στον τομέα (βλ. Παράρτημα Β').

Το 2015 υπήρξε αναζωπύρωση του ενδιαφέροντος για υβριδικά μοντέλα. Προτάθηκαν αλγόριθμοι που υποστηρίζουν offline ενημερώσεις και online ερωτήματα για απλή και 2-ακμική συνεκτικότητα με χρόνο $O(\log n)$ ανά λειτουργία [13]. Την ίδια χρονιά, προτάθηκε και ένας αλγόριθμος για το online πρόβλημα του Ε.Σ.Δ. με πολυπλοκότητα $O(\log^4 n)$ [14].

Το 2022 παρουσιάστηκε ο έως τώρα ταχύτερος αλγόριθμος για απλή online συνεκτικότητα, με χρόνο $O(\log n(\log \log n)^2)$ ανά λειτουργία [15].

Τέλος, το 2025 προτάθηκε αλγόριθμος για την 2-κομβική online συνεκτικότητα με πολυπλοκότητα $\tilde{O}(\log^2 n)$, ακολουθώντας την ίδια πολυλογαριθμική προσέγγιση με την αντίστοιχη ακμική έκδοση [16]. Το $\tilde{O}(\cdot)$ συμβολίζει ότι αγνοούνται πολυλογαριθμικοί πα-

ράγοντες — συνήθως δυνάμεις του $\log n$ — στην εκτίμηση της πολυπλοκότητας, δηλαδή $\tilde{O}(f(n)) = O(f(n) \cdot \log^k n)$ για κάποια σταθερά $k \geq 0$.

Συνολικά, η έρευνα έχει εστιάσει κυρίως στα online προβλήματα δυναμικής συνεκτικότητας. Παρά την ύπαρξη σχεδόν βέλτιστων θεωρητικών αλγορίθμων, οι απαιτήσεις σε κατανόηση, ανάλυση και αποδοτική υλοποίηση παραμένουν υψηλές, καθιστώντας πολλές από τις προτεινόμενες λύσεις μη πρακτικές σε πραγματικά συστήματα.

1.2 Αντικείμενο της Εργασίας

Αντίθετα από το online, το offline πρόβλημα προσφέρει ένα πιο εφικτό και υλοποιήσιμο πλαίσιο από τη φύση του ως μια περίπτωση του online προβλήματος, επιτρέποντας την ανάπτυξη απλών και αποτελεσματικών αλγορίθμων με βέλτιστη πολυπλοκότητα.

Η παρούσα εργασία προτείνει έναν αλγόριθμο για το αμιγώς offline πρόβλημα, πλήρως δυναμικής, 2-ακμικής, συνεκτικότητας, ο οποίος συνδυάζει απλότητα και αποδοτικότητα, καθιστώντας τον ιδανικό για πρακτική εφαρμογή. Αντικείμενο μελέτης της αποτελεί το πρόβλημα offline πλήρως δυναμικής ακμικής δισυνεκτικότητας σε γραφήματα, όπως αυτό διατυπώνεται στο θεωρητικό μοντέλο του άρθρου των Peng, Sandlund και Sleator [2].

Το πρόβλημα αφορά τη διαχείριση πλήρως δυναμικών γραφημάτων, όπου δίνεται μια ακολουθία λειτουργιών, που περιέχει όλων των ειδών τις αλλαγές (updates) - τόσο διαγραφές όσο και εισαγωγές ακμών - και ερωτήματα (queries) ακμικής δισυνεκτικότητας, γνωστής εκ των προτέρων, δηλαδή ένα αμιγώς offline πρόβλημα, και ένα αρχικό γράφημα, όπου στόχος είναι η απάντηση όλων των ερωτημάτων βέλτιστα. Η υλοποίηση βασίστηκε στην ακριβή αποτύπωση του θεωρητικού μοντέλου, ενώ πραγματοποιήθηκαν και πειραματικές μελέτες για την επιβεβαίωση της αποδοτικότητας του αλγορίθμου και τη σύγκρισή της με τη θεωρητική του πολυπλοκότητα.

1.3 Οργάνωση της Εργασίας

Η παρούσα εργασία είναι οργανωμένη σε 5 κεφάλαια, καθένα εκ των οποίων επιτελεί διακριτό ρόλο στη συνολική ανάπτυξη και τεκμηρίωση της μελέτης:

- **Κεφ. 2:** Παρουσιάζεται το θεωρητικό υπόβαθρο που αφορά τα δυναμικά γραφήματα και τα βασικά προβλήματα που τα συνοδεύουν. Δίνονται οι απαραίτητες έννοιες και ορισμοί που είναι θεμελιώδεις για την κατανόηση του αντικειμένου της εργασίας.
- **Κεφ. 3:** Περιγράφεται το πρόβλημα το οποίο μελετήθηκε και γίνεται μια εισαγωγή στο θεωρητικό μοντέλο από τον έγινε η παραγωγή του προτεινόμενου αλγορίθμου που υλοποιήθηκε στο πλαίσιο της εργασίας.
- **Κεφ. 4:** Παρουσιάζονται αναλυτικά όλες οι βασικές υπορουτίνες και τα δομικά στοιχεία του εξεταζόμενου αλγορίθμου. Χωρίζονται σε δύο βασικές κατηγορίες: στις υπορουτίνες επεξεργασίας της ακολουθίας λειτουργιών και στις υπορουτίνες επεξεργασίας του γραφήματος, με τεχνικές αραιοποίησης γραφημάτων.

- **Κεφ. 5:** Μετά τη θεωρητική θεμελίωση, παρουσιάζονται δύο αλγόριθμοι για την επίλυση του προβλήματος της ακμικής δισυνεκτικότητας. Ο πρώτος είναι ο προτεινόμενος αλγόριθμος που μελετήθηκε και περιγράφηκε στο προηγούμενο Κεφάλαιο όλες οι υπορουτίνες του. Και δεύτερον, ένας απλός αλγόριθμος αναφοράς για την πειραματική τους σύγκριση.
- **Κεφ. 6:** Παρουσιάζονται τα συγκριτικά πειραματικά αποτελέσματα των δύο αλγορίθμων για διάφορα μεγέθη εισόδου σε μορφή διαγραμμάτων και πινάκων. Η ανάλυση αποσκοπεί στην αξιολόγηση και την επιβεβαίωση τόσο της εγκυρότητας όσο και της αποδοτικότητας του εξεταζόμενου αλγορίθμου, και σε σύγκριση με τα αποτελέσματα του αλγορίθμου αναφοράς.
- **Κεφ. 7:** Στον επίλογο, προτείνονται πιθανές μελλοντικές επεκτάσεις της εργασίας τόσο ως προς το ίδιο το πρόβλημα της 2-ακμικής συνεκτικότητας όσο και ως προς άλλα είδη προβλημάτων συνεκτικότητας.

Μέρος I

Θεωρητικό Μέρος

Κεφάλαιο 2

Θεωρητικό υπόβαθρο

Στο κεφάλαιο αυτό παρουσιάζεται το απαραίτητο θεωρητικό υπόβαθρο που σχετίζεται με το αντικείμενο της παρούσας εργασίας. Εισάγονται βασικές έννοιες των δυναμικών γραφημάτων, γίνεται διάκριση μεταξύ offline και online εκδοχών των προβλημάτων, και αναλύονται τα προβλήματα συνδεσιμότητας σε αυτό το πλαίσιο. Επιπλέον, περιγράφονται οι βασικοί αλγόριθμοι και τεχνικές για τον υπολογισμό δυσυνεκτικών ακμικών συνιστωσών, τόσο σε στατικά όσο και σε δυναμικά γραφήματα, προετοιμάζοντας το έδαφος για την αλγοριθμική προσέγγιση που ακολουθεί στα επόμενα κεφάλαια.

Για λόγους απλοποίησης, κάθε γράφημα θεωρείται απλό και μη κατευθυνόμενο.

2.1 Δυναμικά Γραφήματα

2.1.1 Στατικά και Δυναμικά Γραφήματα

Τα γραφήματα αποτελούν θεμελιώδες εργαλείο στη θεωρία γράφων, καθώς και σε πλήθος εφαρμογών που σχετίζονται με την επίλυση πρακτικών προβλημάτων. Ένα γράφημα ορίζεται ως ένα ζεύγος $G = (V, E)$ [17, 18], όπου:

- V : το σύνολο των κορυφών,
- $E \subseteq V \times V$: το σύνολο των ακμών, οι οποίες συνδέουν ζεύγη κορυφών.

Τα γραφήματα ενδέχεται να φέρουν βάρη (ή κόστη) στις ακμές. Στην περίπτωση αυτή, εισάγεται μια συνάρτηση βάρους $w(e)$, που περιέχει τις αντίστοιχες τιμές βαρών.

Αναλόγως της συμπεριφοράς τους στον χρόνο, τα γραφήματα κατατάσσονται σε *στατικά* και *δυναμικά*:

- *Στατικά γραφήματα*: η δομή του γραφήματος (κορυφές, ακμές και βάρη) παραμένει σταθερή κατά τη διάρκεια της επεξεργασίας.
- *Δυναμικά γραφήματα*: η δομή μπορεί να μεταβάλλεται με την πάροδο του χρόνου, επιτρέποντας εισαγωγές ή διαγραφές κορυφών και ακμών, καθώς και μεταβολές στα βάρη τους.

Τα δυναμικά γραφήματα είναι κατάλληλα για την ανάλυση συστημάτων που εξελίσσονται χρονικά, όπως τα κοινωνικά δίκτυα ή τα δίκτυα τηλεπικοινωνιών [19, 20].

2.1.2 Λειτουργίες

Σε δυναμικά γραφήματα, η κύρια επεξεργασία βασίζεται σε μια ακολουθία *λειτουργιών* (operations), οι οποίες χωρίζονται σε δύο κατηγορίες:

- *Αλλαγές* (updates): Τροποποιήσεις στη δομή του γραφήματος, όπως εισαγωγές ή διαγραφές ακμών, κορυφών [21], ή αλλαγές βαρών. Συνήθως, οι αλλαγές περιορίζονται στις ακμές για λόγους απλότητας.
- *Ερωτήματα* (queries): Ερωτήσεις που σχετίζονται με τη δομή ή τις ιδιότητες του γραφήματος, όπως το αν δύο κορυφές είναι 2-ακμικά συνδεδεμένες.

2.1.3 Πλήρως και Μερικώς Δυναμικά Προβλήματα

Ανάλογα με το είδος των αλλαγών που επιτρέπονται, τα δυναμικά προβλήματα διακρίνονται στις εξής κατηγορίες:

- *Πλήρως δυναμικά* (fully dynamic): Επιτρέπονται τόσο εισαγωγές όσο και διαγραφές ακμών.
- *Μερικώς δυναμικά* (partially dynamic): Επιτρέπεται μόνο ένα είδος τροποποίησης. Ειδικότερα:
 - *Αύξοντα* (incremental): Επιτρέπονται μόνο εισαγωγές ακμών [22].
 - *Φθίνοντα* (decremental): Επιτρέπονται μόνο διαγραφές ακμών [23].

Η παρούσα εργασία επικεντρώνεται σε πλήρως δυναμικά προβλήματα, επιτρέποντας κάθε είδους τροποποίηση στο γράφημα.

2.1.4 Online και Offline

Η χρονική γνώση των λειτουργιών επηρεάζει σημαντικά τη φύση του προβλήματος. Διακρίνονται δύο βασικές περιπτώσεις:

- *Online*: Η ακολουθία των λειτουργιών δεν είναι γνωστή εκ των προτέρων και αποκάλυπεται σταδιακά κατά την εκτέλεση του αλγορίθμου.
- *Offline*: Η πλήρης ακολουθία των λειτουργιών είναι γνωστή πριν την έναρξη της εκτέλεσης και αποτελεί μέρος της εισόδου του προβλήματος.

Συνδυάζοντας αυτές τις έννοιες με την κατηγοριοποίηση των λειτουργιών, προκύπτουν διαφορετικά είδη δυναμικών προβλημάτων:

- *Αμιγώς online* [8, 10, 15, 24, 14, 16]: Τόσο οι αλλαγές όσο και τα ερωτήματα αποκάλυπτονται σειριακά κατά την εκτέλεση. Αποτελεί την πιο γενική και δύσκολη περίπτωση.
- *Offline αλλαγές και Online ερωτήματα* [13]: Η σειρά των αλλαγών είναι γνωστή εξ αρχής, ενώ τα ερωτήματα εμφανίζονται σταδιακά.
- *Αμιγώς offline* [2, 3]: Όλες οι λειτουργίες είναι γνωστές εκ των προτέρων. Η πιο απλοποιημένη μορφή του προβλήματος.

Η γνώση μέρους ή όλης της εισόδου επιτρέπει πιο αποδοτικές προεπεξεργασίες και σχεδιασμό εξειδικευμένων και, ταυτόχρονα, πιο απλών δομών δεδομένων. Έτσι, τα offline προβλήματα συχνά αποτελούν χρήσιμο σημείο αναφοράς για την αξιολόγηση ή προσέγγιση των online εκδοχών τους.

Αξιοσημείωτο είναι ότι, όπως δείχνει και η εργασία [2], στα k -ακμικά ή k -κομβικά συνεκτικά προβλήματα, όσο αυξάνεται το k , η απόσταση σε απόδοση μεταξύ των γνωστών offline και online αλγορίθμων αυξάνεται σημαντικά.

Η παρούσα εργασία θα απασχοληθεί με το αμιγώς offline πρόβλημα.

2.2 Συνεκτικότητα

2.2.1 Ακμική και Κομβική Συνεκτικότητα

Μια θεμελιώδης έννοια στη θεωρία γράφων είναι η έννοια της συνεκτικότητας. Ένα γράφημα θεωρείται συνεκτικό όταν κάθε ζεύγος κορυφών συνδέεται μέσω τουλάχιστον ενός μονοπατιού. Συνεπώς, όλες οι κορυφές ενός συνεκτικού γραφήματος είναι προσβάσιμες ανά μεταξύ τους, δηλαδή ότι δεν υπάρχουν απομονωμένες ομάδες κορυφών [17, 18].

Τα προβλήματα συνεκτικότητας χωρίζονται σε δύο βασικές κατηγορίες, την *ακμική* και την *κομβική* συνεκτικότητα. Η *ακμική* συνεκτικότητα αναφέρεται στην ικανότητα του γραφήματος να παραμένει συνεκτικό ακόμα και μετά από την αφαίρεση κάποιων ακμών, ενώ πλήρως αντίστοιχα συμβαίνει για τη *κομβική* συνεκτικότητα.

Πιο αναλυτικά, οι έννοιες της είτε k -ακμικής είτε k -κομβικής συνεκτικότητας αναφέρονται στην ικανότητα ενός γραφήματος να παραμένει συνεκτικό ακόμα και μετά την αφαίρεση $k-1$ ακμών ή κορυφών αντίστοιχα. Κατά τον ορισμό, ένα γράφημα είναι k -ακμικά συνεκτικό εάν για οποιοδήποτε ζεύγος κορυφών, αν αφαιρεθεί οποιοδήποτε σύνολο ακμών λιγότερων από k , οι κορυφές παραμένουν συνδεδεμένες μεταξύ τους μέσω τουλάχιστον ενός μονοπατιού. Αντίστοιχα και για την k -κομβική συνεκτικότητα, αν αφαιρεθεί οποιοδήποτε σύνολο κορυφών λιγότερων από k , οι κορυφές παραμένουν συνδεδεμένες μεταξύ τους μέσω τουλάχιστον ενός μονοπατιού.

2.2.2 Ερωτήματα Συνεκτικότητας

k -ακμική συνεκτικότητα

Οι πλήρως δυναμικοί αλγόριθμοι γραφημάτων για k -ακμική συνεκτικότητα υποστηρίζουν το τρίπτυχο των λειτουργιών:

- $insert(u, v)$: εισαγωγή ακμής (u, v)
- $delete(u, v)$: διαγραφή ακμής (u, v)
- $query(u, v)$: έλεγχος αν οι κορυφές u και v είναι k -ακμικά συνεκτικές (δηλ., ανήκουν στην ίδια k -ακμικά συνεκτική συνιστώσα)

Δύο κορυφές $u, v \in V$ λέγονται k -ακμικά συνεκτικές αν και μόνο αν ισχύει:

$$\lambda(u, v) = \min_{\substack{S \subseteq E \\ u \leftrightarrow v \text{ στο } G-S}} |S| \geq k,$$

όπου $\lambda(u, v)$ δηλώνει τον ελάχιστο αριθμό ακμών των οποίων η αφαίρεση αποσυνδέει την u από τη v , και $G - S$ είναι το γράφημα που προκύπτει μετά την αφαίρεση των ακμών του συνόλου S [25, 26]. Δηλαδή, δύο κορυφές που είναι k -ακμικά συνεκτικές, παραμένουν συνεκτικές ακόμα και μετά τη διαγραφή οποιωνδήποτε $k-1$ ακμών. Οπότε ένα ερώτημα $q(u, v)$ επιστρέφει *θετική απάντηση* αν και μόνο αν οι κορυφές u και v είναι k -ακμικά συνεκτικές, δηλαδή $\lambda(u, v) \geq k$.

2-ακμική συνεκτικότητα

Η παρούσα εργασία επικεντρώνεται αποκλειστικά στην περίπτωση των ερωτημάτων 2-ακμικής συνεκτικότητας. Δηλαδή, για δύο κορυφές $u, v \in V$, λέμε ότι είναι 2-ακμικά συνεκτικές αν και μόνο αν ισχύει μία από τις ισοδύναμες παρακάτω συνθήκες:

- Υπάρχουν τουλάχιστον δύο ακμικά ξένα (edge-disjoint) μονοπάτια που συνδέουν την u με τη v .
- Οι κορυφές u και v ανήκουν στην ίδια ακμική δισυνεκτική συνιστώσα του γραφήματος.

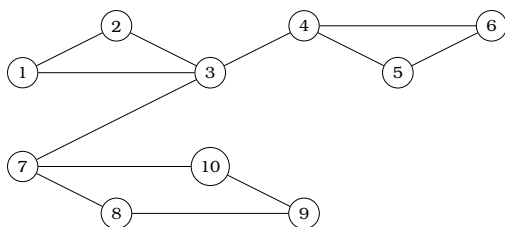
2.3 Υπολογισμός Δισυνεκτικών Δυνιστωσών

Σε αυτό το κεφάλαιο θα αναλυθούν περιληπτικά οι μέχρι σήμερα τρόποι επίλυσης βασικών συνεκτικών προβλημάτων για στατικά και δυναμικά γραφήματα. Σε μια πρώτη ανάλυση, όπως και πριν, οι δισυνεκτικές συνιστώσες χωρίζονται σε δύο είδη προβλημάτων, τις ακμικές και τις κομβικές, σε πλήρη αντιστοιχία με τον ορισμό προηγουμένως.

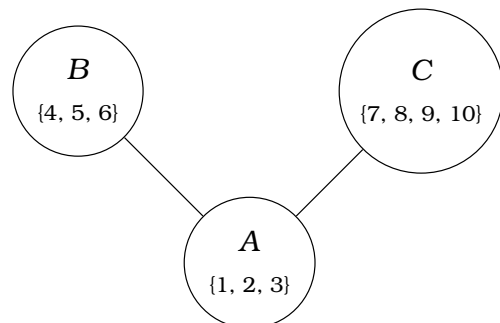
2.3.1 Ακμική Δισυνεκτικότητα

Για στατικά γραφήματα, είναι γνωστοί οι γραμμικοί αλγόριθμοι για το πρόβλημα της απλής συνεκτικότητας και της δισυνεκτικότητας ακμών, βασισμένοι στην ιδέα της εύρεσης των γεφυρών ενός γραφήματος. Γέφυρα χαρακτηρίζεται μία ακμή που αν διαγραφεί αποσυνδέεται ένα μέρος του γραφήματος από το υπόλοιπο. Αν σε ένα μη κατευθυνόμενο γράφημα δεν υπάρχουν γέφυρες, τότε το γράφημα μπορεί να συμπυκνωθεί σε μια δισυνεκτική ακμική συνιστώσα, δηλαδή μια συνιστώσα κορυφών όπου κάθε κορυφή του γραφήματος συνδέεται με όλες τις υπόλοιπες με τουλάχιστον δύο ακμικά ξένα μονοπάτια.

Από την άλλη, αν υπάρχουν γέφυρες, τότε το γράφημα μπορεί νοητικά να συρρικνωθεί σε ένα δενδρικό γράφημα και να χωριστεί σε δισυνεκτικές ακμικές συνιστώσες που ενώνονται μεταξύ τους μέσω των γεφυρών. Για να απαντηθεί ένα ερώτημα δισυνεκτικότητας μεταξύ δύο κορυφών σε στατικό γράφημα, αρκεί να υπολογιστεί το δενδρικό γράφημα με την εύρεση των δισυνεκτικών ακμικών συνιστωσών και ο αντιπρόσωπος των δύο κορυφών να είναι κοινός. Δηλαδή να βρίσκονται στην ίδια συνιστώσα.



Σχήμα 2.1: Παράδειγμα ενός απλού μη-κατευθυνόμενου γραφήματος



Σχήμα 2.2: Αναπαράσταση του Παραγόμενου Δενδρικού Γραφήματος

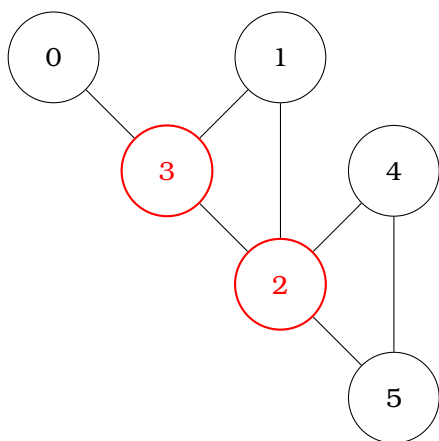
Η εύρεση του δένδρου γεφυρών, δηλαδή της εύρεσης των γεφυρών και του μετασχηματισμού του γραφήματος στο ισοδύναμο δενδρικό του γίνεται με έναν αλγόριθμο του Tarjan

[27, 28], όπου βασίζεται πάνω στον στη διερεύνηση πρώτα κατά βάθος (depth-first search - DFS). Ένα παράδειγμα φαίνεται στο Σχήμα 2.1-2.2.

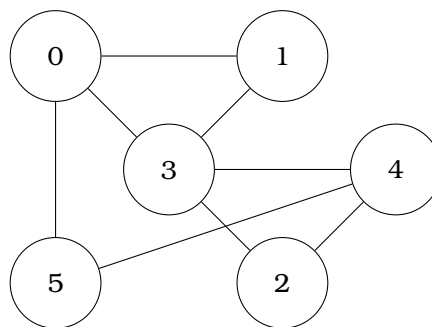
2.3.2 Κομβική Δισυνεκτικότητα

Το δισυνεκτικό κομβικό πρόβλημα, από την άλλη, βασίζεται στην ύπαρξη ενός χαρακτηρισμού κορυφών, ως *αρθρώσεις* (articulation points), όπου η αφαίρεσή τους προκαλεί αποσύνδεση του γραφήματος, αντίστοιχη έννοια με τις γέφυρες [17, 18].

Για παράδειγμα, στο συνεκτικό γράφημα που παρουσιάζεται στο Σχήμα 2.3, αν αφαιρεθούν οι κορυφές 2 ή 3, το γράφημα θα διασπαστεί σε δύο συνεκτικές συνιστώσες. Επομένως, οι κορυφές αυτές είναι αρθρώσεις και το γράφημα δεν είναι κομβικά δισυνεκτικό. Εάν προστεθούν όμως οι ακμές $(0, 1)$, $(0, 5)$ και $(2, 4)$, το γράφημα καθίσταται κομβικά δισυνεκτικό, αφού δεν υπάρχουν αρθρώσεις, δηλαδή κορυφές που αν αφαιρεθούν το γράφημα σπάσει η συνεκτικότητά του.



Σχήμα 2.3: Παράδειγμα μη κομβικού δισυνεκτικού γραφήματος. Με κόκκινο φαίνονται οι αρθρώσεις του



Σχήμα 2.4: Παράδειγμα κομβικού δισυνεκτικού γραφήματος

Ο τρόπος επίλυσης αυτού του προβλήματος είναι αρχικά η εύρεση των αρθρώσεων σε ένα γράφημα χρησιμοποιώντας αναζήτηση κατά βάθος (DFS). Ένα γράφημα $G(V, E)$ είναι κομβικά δισυνεκτικό αν και μόνο αν για κάθε κορυφή u του γραφήματος, υπάρχει τουλάχιστον μία πίσω ακμή που βγαίνει από το υποδέντρο που ριζώνει στην κορυφή u προς κάποιον πρόγονο v της κορυφής u . Όταν γίνεται αναφορά σε υποδέντρο ριζωμένο στην κορυφή u , εννοούνται όλοι οι απόγονοι της u (εκτός της ίδιας της κορυφής u). Με άλλα λόγια, όταν γίνει επιστροφή από μια κορυφή u , πρέπει να διασφαλιστεί ότι υπάρχει μια πίσω ακμή από κάποιον απόγονο της u προς κάποιον πρόγονο (γονέα ή ανώτερο) της u . Υπάρχει μια εξαίρεση για τον ριζικό κόμβο του δέντρου. Εάν ο ριζικός κόμβος έχει περισσότερους από έναν απογόνους, τότε θεωρείται άρθρωση [28].

Κεφάλαιο 3

Offline 2-Ακμική Συνεκτικότητα

Στο κεφάλαιο αυτό παρουσιάζεται αρχικά η ακριβής περιγραφή του προβλήματος και γίνεται εισαγωγή στις βασικές έννοιες που θα χρησιμοποιηθούν εκτενώς κατά την ανάλυση του αλγοριθμικού μοντέλου επίλυσης που αναπτύσσεται στα επόμενα κεφάλαια της εργασίας.

Αναλύονται διεξοδικά έννοιες όπως ενεργές κορυφές (active vertices), μόνιμες ακμές (permanent edges) και κορυφές αντιπρόσωποι (representative vertices). Στη συνέχεια, παρουσιάζεται η κεντρική ιδέα του αλγορίθμου, καθώς και οι βασικές βοηθητικές υπο-ρουτίνες που τη συνοδεύουν.

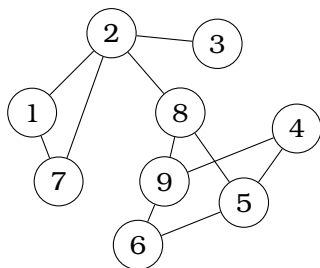
3.1 Ορισμοί

3.1.1 Περιγραφή του Προβλήματος

Έστω ένα αρχικό γράφημα $G(V, E)$ με V το σύνολο των κορυφών του και E το σύνολο των ακμών του, και μία ακολουθία λειτουργιών που είναι γνωστή εκ των προτέρων (offline) και αποτελείται από τις εξής:

- $\text{insert}(u, v)$ ή $I(u, v)$: εισαγωγή ακμής (u, v)
- $\text{delete}(u, v)$ ή $D(u, v)$: διαγραφή ακμής (u, v)
- $\text{query}(u, v)$ ή $Q(u, v)$: έλεγχος αν οι u, v είναι 2-ακμικά συνδεδεμένες

Σκοπός του προβλήματος είναι η απάντηση όλων των ερωτημάτων σε βέλτιστο συνολικό χρόνο $O(t \cdot \log n)$, σύμφωνα με το γνωστό κάτω φράγμα [11], όπου t είναι το πλήθος των λειτουργιών και n το πλήθος των αρχικών κορυφών του γραφήματος.



1	2	3	4	...	60	61	...
$I(5,6)$	$D(4,3)$	$Q(2,1)$	$I(5,4)$	...	$D(8,9)$	$I(2,5)$	...

Σχήμα 3.1: Παράδειγμα εισόδου με: 1. αρχικό γράφημα και, 2. ακολουθία λειτουργιών.

3.1.2 Απλοποίηση Δεδομένων Εισόδου

Για λόγους απλοποίησης, μια ισοδύναμη είσοδος είναι να δίνεται ένα κενό γράφημα $G(V, E = \emptyset)$ με γνωστό το πλήθος των κορυφών του και μια ακολουθία από λειτουργίες στις οποίες συμπεριλαμβάνονται στην αρχή της ακολουθίας και όλες οι ακμές του αρχικού γραφήματος ως εισαγωγές ακμών.

1	2	3	...	11	12	13	14	...	71	72	...
I(1,2)	I(3,2)	I(4,5)	...	I(8,9)	I(5,6)	D(4,3)	G(2,1)	...	D(8,9)	I(2,5)	...

Σχήμα 3.2: Παραλληλαγή εισόδου: Εκτεταμένη ακολουθία λειτουργιών από το Σχήμα 3.1.

3.1.3 Θεωρητικό Μοντέλο

Στην παρούσα μελέτη αναλύεται το υπολογιστικό μοντέλο (*framework*) των R. Peng, B. Sandlund & D. Sleator[2], όπως αναφέρθηκε και αρχικά, για την επίλυση offline προβλημάτων συνεκτικότητας, το οποίο έχει εφαρμοστεί επιτυχώς σε διάφορες περιπτώσεις.

Η βασική προσέγγιση στηρίζεται στη στρατηγική *διαίρει και βασίλευε*, όπου η ακολουθία λειτουργιών διασπάται σε μικρότερα, διαχειρίσιμα υποπροβλήματα. Σε κάθε στάδιο, δημιουργείται ένα ισοδύναμο γράφημα, αξιοποιώντας τεχνικές αραιοποίησης (*sparsification*) [4, 9, 5, 22], οι οποίες μειώνουν σημαντικά την πληροφορία που απαιτείται ανά αναδρομική κλήση. Μέσω αυτής της διαδικασίας επιτυγχάνεται συνολικά το βέλτιστο κάτω φράγμα, όπως αυτό έχει οριστεί από τους Patrascu & Demaine [11] (βλ. Παράρτημα Β). Δηλαδή, πετυχαίνει συνολικό χρόνο εκτέλεσης $O(t \cdot \log n)$, όπου n είναι το πλήθος των αρχικών κορυφών του γραφήματος και t το πλήθος των λειτουργιών.

3.1.4 Ανάλυση Χρονικής Πολυπλοκότητας

Στην ανάλυση του μοντέλου, όπου t αντιπροσωπεύει το μέγεθος της ακολουθίας λειτουργιών, προκύπτει η εξής σχέση αναδρομής για τον χρόνο εκτέλεσης [2]:

$$T(t) = T\left(\frac{t}{2}\right) + O(t) \Rightarrow T(t) = O(t \cdot \log t).$$

Η πολυπλοκότητα αυτή ισχύει υπό την προϋπόθεση ότι, για κάθε υποπρόβλημα μεγέθους t , ότι οι επιμέρους υπορουτίνες:

- για την επεξεργασία και την αραιοποίηση (*sparsification*) του γραφήματος στην εκάστοτε αναδρομική κλήση, και
- για την επεξεργασία και εξαγωγή πληροφοριών από τις εκάστοτε λειτουργίες

θα έχουν γραμμική πολυπλοκότητα ως προς το μέγεθος της εισόδου τους και το παραγόμενο γράφημα θα έχει μέγεθος $O(l)$, όπου l το πλήθος των κορυφών που χρησιμοποιούνται από τις λειτουργίες, όπου καλούνται παρακάτω ως *ενεργές*.

Ωστόσο, η αποδοτικότητα του αλγορίθμου διαφοροποιείται ανάλογα με τη σχέση του πλήθους λειτουργιών t σε σχέση με το πλήθος των αρχικών κορυφών n .

πολυωνυμικά φραγμένο t ως προς n

Αν t είναι πολυωνυμικά φραγμένο ως προς n ($t = O(n^c)$) για κάποια σταθερά c , τότε ισχύει ότι:

$$T(t) = O(t \cdot \log n), \quad \text{βέλτιστη}$$

εκθετικά φραγμένο t ως προς n

Αν όχι, δηλαδή t είναι εκθετικό ως προς n , τότε ο χρόνος εκτέλεσης αυξάνεται σε:

$$T(t) = O(t \cdot \log t) \gg O(t \cdot \log n), \quad \text{μη βέλτιστη}$$

Σε αυτή την περίπτωση, η πολυπλοκότητα παύει να είναι βέλτιστη, αφού υπάρχουν γνωστοί διαδικτυακοί (online) αλγόριθμοι με χρόνο $O(t \cdot \text{polylog}(n))$, που είναι πρακτικά πιο αποδοτικοί.

Για να διατηρηθεί η αποδοτικότητα σε περιπτώσεις πολύ μεγάλου t , είναι δυνατή η εφαρμογή στρατηγικής ομαδοποίησης: η ακολουθία των λειτουργιών $x_1, x_2, \dots, x_t$ χωρίζεται σε μπλοκ μεγέθους n^2 . Δεδομένου ότι κανένα μπλοκ δεν περιέχει γράφημα μεγαλύτερου μεγέθους από $O(n^2)$, κάθε υποπρόβλημα μπορεί να επιλυθεί ανεξάρτητα. Συνολικά, επιτυγχάνεται χρόνος:

$$T(t) = O(t \cdot \log n).$$

Η τεχνική αυτή καθιστά τον αλγόριθμο αποδοτικό και εφαρμόσιμο ακόμη και σε περιπτώσεις με πολύ μεγάλες ακολουθίες λειτουργιών.

Σημείωση

Η παραπάνω στρατηγική δεν υλοποιήθηκε στην παρούσα εργασία και δεν εξετάστηκε πειραματικά η πρακτική της επίδραση. Θεωρήθηκε δεδομένο ότι το πλήθος των λειτουργιών είναι πολυωνυμικά φραγμένο ως προς το μέγεθος του αρχικού γραφήματος.

Βασικές Υπορουτίνες

Στο κεφάλαιο αυτό παρουσιάζονται οι βασικοί αλγόριθμοι που αποτελούν τα δομικά στοιχεία του αλγοριθμικού μοντέλου για την επίλυση του offline ακμικού δισυνεκτικού προβλήματος. Οι υπορουτίνες αυτές απασχολούνται αρχικά με την διαχείριση της ακολουθίας των λειτουργιών, καθορίζοντας τον τρόπο με τον οποίο θα εκτελεστούν τα βήματα της επίλυσης. Κατά δεύτερον, στη διαχείριση της ανάλυσης και της επεξεργασίας του δοθέντος γραφήματος, εξασφαλίζοντας την κατάλληλη προετοιμασία της δομής του για την εφαρμογή της λύσης, παράγοντας αυτό που παρακάτω καλείται *ισοδύναμο γράφημα*.

4.1 Επεξεργασία Ακολουθίας Λειτουργιών

4.1.1 Ενεργές Κορυφές

Δοθέντος ενός χρονικού διαστήματος λειτουργιών όλες οι κορυφές που εμπλέκονται έστω και σε μία λειτουργία στο εν λόγω διάστημα ονομάζονται *ενεργές* (active), ενώ οι υπόλοιπες χαρακτηρίζονται ως *ανενεργές* (inactive). Έστω μία ακολουθία από λειτουργίες. Τότε μπορεί να απαντηθεί σε γραμμικό χρόνο, ως προς το μήκος της ακολουθίας, για κάθε κορυφή ποιες κορυφές είναι *ενεργές* (active). Δείτε το παράδειγμα του Πίνακα 4.1.

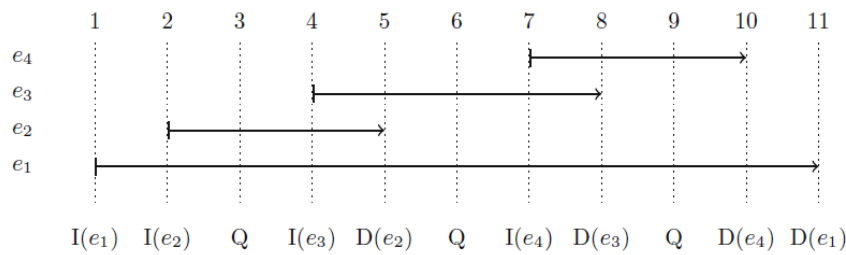
Χρόνος	...	5	6	7	8	9	10	...
Διάστημα	...	$I(1, 3)$	$D(3, 9)$	$\mathcal{Q}(2, 3)$	$D(9, 2)$	$\mathcal{Q}(1, 9)$	$I(1, 2)$	...

Πίνακας 4.1: *Ενεργές κορυφές: {1,2,3,9}*

Ο πιο απλός και αποδοτικός τρόπος για τον υπολογισμό τους είναι ένα απλό *γραμμικό πέρασμα* στο δοθέν διάστημα των λειτουργιών, όπου όποια κορυφή βρίσκεται σημειώνεται ως ενεργή. Όλες οι υπόλοιπες που δεν χρησιμοποιήθηκαν θεωρούνται ανενεργές για αυτό το διάστημα.

4.1.2 Διάρκεια Ζωής μίας Ακμής

Κάθε ακμή e έχει μία περίοδο ζωής $[I(e), D(e)]$, όπου $I(e)$ είναι η χρονική στιγμή εισαγωγής της και $D(e)$ η χρονική στιγμή διαγραφής της. Για παράδειγμα (βλ. Σχήμα 4.1, μία ακμή e με $I_e = 4$ και $D_e = 8$ έχει περίοδο ζωής το διάστημα $[4, 8]$.



Σχήμα 4.1: Παράδειγμα διάρκειας ζωής ακμών

4.1.3 Μόνιμες Ακμές

Ορισμός

Έστω ένα διάστημα $[l, r]$, το οποίο αναφέρεται σε χρονικό υποδιάστημα εντός της συνολικής ακολουθίας λειτουργιών. Οι ακμές κατηγοριοποιούνται ως εξής:

- **Μόνιμες ακμές** (*permanent*): Πρόκειται για ακμές που είναι ενεργές καθ' όλη τη διάρκεια του διαστήματος $[l, r]$, δηλαδή ισχύει:

$$I(e) \leq l \leq r \leq D(e)$$

Με άλλα λόγια, η ακμή έχει εισαχθεί πριν ή στην αρχή του διαστήματος και δεν έχει αφαιρεθεί μέχρι το τέλος του. Οι ακμές αυτές δεν τροποποιούνται καθόλου εντός του διαστήματος.

- **Μη μόνιμες ακμές** (*non-permanent*): Αυτές είναι οι ακμές των οποίων η περίοδος ζωής επικαλύπτει μερικώς το διάστημα $[l, r]$, αλλά δεν το καλύπτει πλήρως. Δηλαδή, ισχύει ότι:

$$I(e) \in (l, r) \quad \text{ή} \quad D(e) \in (l, r)$$

Στην περίπτωση αυτή, η ακμή εισάγεται ή αφαιρείται εντός του διαστήματος και επομένως η κατάστασή της τροποποιείται κατά τη διάρκειά του.

Αυτή η ταξινόμηση είναι κρίσιμη για την κατανόηση των ακμών που παραμένουν αμετάβλητες σε ένα χρονικό παράθυρο, έναντι εκείνων που υφίστανται μεταβολές και πιθανώς επηρεάζουν την τοπολογία του γραφήματος κατά την εξέλιξη των λειτουργιών.

Υπολογισμός

Για τον υπολογισμό τους χρειάζεται πρώτα να γίνει μια προεπεξεργασία (preprocessing) στην ακολουθία λειτουργιών, ώστε για κάθε λειτουργία προσθήκης ή αφαίρεσης ακμής να υπάρχει μαζί και το πότε αυτές αφαιρούνται ή προστίθενται από το ή στο γράφημα, αντίστοιχα. Δηλαδή, χρειάζεται για κάθε ακμή e να χρησιμοποιηθεί η πληροφορία της διάρκειας ζωής της, $[I(e), D(e)]$, και να αποθηκευτεί πάνω στην στιγμή της λειτουργίας της εισαγωγής ή της διαγραφής.

Χρειάζεται επομένως ένας αποδοτικός τρόπος που να παίρνει μία ακολουθία από λειτουργίες και να επιστρέφει το ζεύγος της εκάστοτε λειτουργίας επαυξημένο με έναν ακόμη αριθμό που αντιπροσωπεύει τη διάρκεια ζωής της ακμής. Δίνεται το παρακάτω παράδειγμα, όπου

παρατηρείται ότι πχ. η ακμή $e_{1,3}$ έχει διάρκεια $[5, 10]$ και στην επαυξημένη ακολουθία της στην στιγμή 5 της εισαγωγής της ($I(5, 10)$) καταγράφεται και η στιγμή 10 όπου αναφέρεται στην αφαίρεσή της ($D(5, 10)$):

...	5	6	7	8	9	10	...
...	$I(1, 3)$	$I(3, 9)$	$Q(2, 3)$	$D(9, 3)$	$I(1, 2)$	$D(1, 3)$	...

Πίνακας 4.2: Παράδειγμα αρχικής ακολουθίας λειτουργιών

...	5	6	7	8	9	10	...
...	$(I(1, 3), 10)$	$(I(3, 9), 8)$	$(Q(2, 3), -)$	$(D(9, 3), 5)$	$(I(1, 2), 24)$	$(D(1, 3), 5)$	...

Πίνακας 4.3: Επαυξημένη τελική ακολουθία λειτουργιών

Προεπεξεργασία Λειτουργιών

Υπάρχουν δύο αποδοτικοί τρόποι επίλυσης του προβλήματος. Περιγράφονται οι αλγόριθμοι ως εξής:

- **με ταξινόμηση:** Πρόκειται για έναν αλγόριθμο με θεωρητική πολυπλοκότητα $O(t \cdot \log t)$, ο οποίος περιλαμβάνει τα εξής στάδια:
 - Σε κάθε λειτουργία αποδίδεται ο χρονικός δείκτης εμφάνισής της.
 - Μέσω ενός γραμμικού περάσματος, αφαιρούνται όλες οι λειτουργίες που αντιστοιχούν σε ερωτήματα.
 - Η ακολουθία των υπολειπόμενων λειτουργιών ταξινομείται βάσει των ζευγών κορυφών, έτσι ώστε για κάθε ζεύγος (u, v) ή ακμή $e_{u,v}$, η εισαγωγή και η διαγραφή (εφόσον υπάρχουν) να βρίσκονται σε διαδοχικές θέσεις. Αυτό επιτρέπει τον εντοπισμό, σε γραμμικό χρόνο, της διάρκειας ζωής κάθε ακμής και καταγραφή της πάνω στις ίδιες τις λειτουργίες της ακολουθίας.
 - παραγωγή σε γραμμικό χρόνο της επαυξημένης ακολουθίας λειτουργιών συμπεριλαμβανομένων και των ερωτημάτων συνεκτικότητας.

Όπως αναλύθηκε προηγουμένως στην ενότητα ανάλυσης πολυπλοκότητας 3.1.4, ισχύει ότι $O(t \cdot \log t) \gg O(t \cdot \log n)$, αν t εκθετικά φραγμένο ως προς n . Ωστόσο, υπό την υπόθεση ότι το t είναι πολυωνυμικά φραγμένο ως προς το n στην παρούσα εργασία, ο συγκεκριμένος αλγόριθμος θεωρείται ασυμπτωτικά βέλτιστος.

- **με χρήση hash map:** Ένας αποδοτικός αλγόριθμος με γραμμική *αποσβεστική* (amortized) χρονική πολυπλοκότητα $O(t)$, ο οποίος χρησιμοποιεί έναν χάρτη κατακερματισμού (hash map) και να εξασφαλίζει προσπέλαση σε χρόνο $O(1)$ για τις χρονικές στιγμές εισαγωγής και διαγραφής των ακμών.

Στην πράξη, όπου καθοριστικής σημασίας είναι η ευελιξία και η ευκολία υλοποίησης, η επιλογή της χρήσης αλγορίθμου με βάση δομή κατακερματισμού (hash map) αποτελεί μια πρακτική και αποτελεσματική λύση, χωρίς σημαντικό κόστος στον συνολικό χρόνο εκτέλεσης. Για τον λόγο αυτό, στην τελική υλοποίηση του προτεινόμενου αλγορίθμου επιλέχθηκε η δεύτερη προσέγγιση.

Αλγόριθμος Εύρεσης

Με δεδομένη την επαυξημένη ακολουθία λειτουργιών, για κάθε υποδιάστημα ενός αρχικού διαστήματος μπορεί να βρεθεί σε γραμμικό χρόνο ποιες ακμές είναι μόνιμες. Στο παράδειγμα του Πίνακα 4.3, για το υποδιάστημα $[6, 8]$, η μόνιμη ακμή είναι η $e_{1,3}$, αφού η προσθήκη της είναι πριν το υπο-διάστημα και η αφαίρεση μετά.

Έστω αρχικό χρονικό διάστημα λειτουργιών $[l, r]$. Το υποδιάστημα στο οποίο θα υπολογιστούν οι μόνιμες ακμές, έστω $[i, j]$, για κάθε ακμή e με χρονικό διάστημα ζωής $[I(e), D(e)]$ θεωρείται μόνιμη εάν πληρούνται οι συνθήκες:

$$l \leq I(e) < i \quad \wedge \quad j \leq D(e) < r,$$

Η άμεση εύρεση των μόνιμων ακμών για κάποιο υποδιάστημα είναι δύσκολη και υπολογιστικά κοστοβόρα χωρίς την κατάλληλη προ-επεξεργασία των λειτουργιών. Για το λόγο αυτό, η υλοποίηση βασίζεται σε τεχνικές προεπεξεργασίας που επιτρέπουν την αποδοτική ανάκτηση των ακμών που πληρούν τις παραπάνω προϋποθέσεις, αξιοποιώντας τις χρονικές στιγμές εισαγωγής $I(e)$ και διαγραφής $D(e)$ της κάθε ακμής.

4.2 Επεξεργασία Γραφήματος (Ισοδύναμο Γράφημα)

Το υποκεφάλαιο αυτό παρουσιάζει όλες τις υπορουτίνες που χρησιμοποιούνται για την παραγωγή του *ισοδύναμου γραφήματος* ως μια αποδοτική μέθοδος αραιοποίησης του γραφήματος δεδομένης μιας ακολουθίας λειτουργιών.

4.2.1 Αντιπρόσωποι

Κορυφή αντιπρόσωπος είναι μια κορυφή που αναπαριστά κάποια ακμική δισυνεκτική συνιστώσα του γραφήματος. Λόγω της δυναμικής φύσης του γραφήματος, σε κάθε χρονική *κατάσταση* (state) υπάρχουν κορυφές *αντιπρόσωποι* (representatives), στις οποίες αντιστοιχούν διάφορες κορυφές του αρχικού γραφήματος. Άρα, αν δύο ή περισσότερες κορυφές αντιστοιχούν στην ίδια κορυφή αντιπρόσωπο, τότε ανήκουν στην ίδια ακμική δισυνεκτική συνιστώσα. Επομένως είναι 2-ακμικά συνδεδεμένες.

Ο *πίνακας αντιπροσώπων* (representatives table) είναι μια δομή που καταγράφει τον αντιπρόσωπο κάθε αρχικής κορυφής του γραφήματος, δηλαδή σε ποια ακμική δισυνεκτική συνιστώσα ανήκει κάθε στιγμή.

Αξίζει να σημειωθεί ότι, αν το γράφημα τροποποιηθεί κατά την εκτέλεση του αλγορίθμου, απαιτείται ανανέωση του πίνακα αντιπροσώπων. Για να διατηρείται έγκυρος ο πίνακας, αρκεί μόνο η ενημέρωση των αντιπροσώπων για τις κορυφές ενδιαφέροντος, δηλαδή μόνο τις ενεργές κορυφές. Η ορθή ανανέωση αυτού εξασφαλίζει τη σωστή αντιστοίχιση των κορυφών στις νέες ακμικές δισυνεκτικές συνιστώσες που ενδέχεται να προκύψουν από τις τροποποιήσεις του γραφήματος.

4.2.2 Ακμικές Δισυνεκτικές Συνιστώσες (Δεντρικό Γράφημα)

Αρχικά, είναι απαραίτητη η περιγραφή του αλγορίθμου που χρησιμοποιείται για την εύρεση των γεφυρών (bridges) σε ένα γράφημα [27]. Βασίζεται στην αναζήτηση κατά βάθος (DFS) και εκμεταλλεύεται δύο βασικές έννοιες:

1. Χρόνος Ανακάλυψης (Discovery Time): Ο χρόνος που μια κορυφή ανακαλύπτεται κατά τη διάρκεια του DFS.
2. Χρόνος Χαμηλότερης Επίσκεψης (Low Time): Η μικρότερη τιμή του χρόνου ανακάλυψης που είναι προσβάσιμη από την κορυφή μέσω των γειτονικών της κορυφών.

ΑΛΓΟΡΙΘΜΟΣ 4.1: *Bridges*

```

1: function COMPUTE_BRIDGES(graphG)
2:   Initialize visited, disc, low, and parent arrays
3:   for each vertex  $i$  in graphG do
4:     if  $i$  is not visited then
5:       BRIDGEUTIL( $i$ , visited, parent, low, disc, graphG)
6:     end if
7:   end for
8:   return results
9: end function
10:
11: function BRIDGEUTIL( $u$ , visited, parent, low, disc, graphG)
12:   visited[ $u$ ]  $\leftarrow$  True
13:   disc[ $u$ ]  $\leftarrow$  Time
14:   low[ $u$ ]  $\leftarrow$  Time
15:   Time  $\leftarrow$  Time + 1
16:   for each vertex  $v$  adjacent to  $u$  do
17:     if  $v$  is not visited then
18:       parent[ $v$ ]  $\leftarrow u$ 
19:       BRIDGEUTIL( $v$ , visited, parent, low, disc, graphG)
20:       low[ $u$ ]  $\leftarrow$  min(low[ $u$ ], low[ $v$ ])
21:       if low[ $v$ ] > disc[ $u$ ] then
22:         mark edge( $u$ ,  $v$ ) and edge( $v$ ,  $u$ ) as a bridge
23:       end if
24:     else
25:       if  $v \neq$  parent[ $u$ ] then
26:         low[ $u$ ]  $\leftarrow$  min(low[ $u$ ], disc[ $v$ ])
27:       end if
28:     end if
29:   end for
30: end function

```

Έτσι, ο κύριος τρόπος εύρεσης των ακμικών δισυνεκτικών συνιστωσών (2-edge-connected components) γίνεται μέσω της εύρεσης του *δέντρου γεφυρών* (bridge tree), όπου καταγράφεται σε ποια ακμική δισυνεκτική συνιστώσα ανήκει κάθε κορυφή. Η διαδικασία αυτή διασφαλίζει ότι το γράφημα χωρίζεται σε συνιστώσες που δεν περιλαμβάνουν γέφυρες μεταξύ τους, κάνοντας την ανάλυση του γραφήματος πιο αποτελεσματική [29]. Ο γραμμικός αυτός αλγόριθμος έχει τα εξής βήματα:

- **Εντοπισμός Γεφυρών (Bridge Identification):** Στο πρώτο βήμα, ο αλγόριθμος αναγνωρίζει όλες τις γέφυρες του γραφήματος.
- **Αρχικοποίηση Συνιστωσών (Component Initialization):** Ο αλγόριθμος αρχίζει με την αναγνώριση των συνιστωσών του γραφήματος, χρησιμοποιώντας DFS. Για κάθε κορυφή που δεν την έχει επισκεφθεί, εκτελεί DFS με έναν μοναδικό αριθμό συνιστώσας.
- **Εκτέλεση DFS (DFS Execution):** Η DFS εξετάζει όλες τις γειτονικές κορυφές που δεν έχουν ήδη επισκεφθεί. Όσες συνδέονται μέσω μη-γεφυρών θεωρούνται μέρος της ίδιας συνιστώσας. Αγνοούνται οι γέφυρες στο γράφημα.
- **Αύξηση Αριθμού Συνιστώσας (Component Counter Incrementation):** Μετά από κάθε DFS ακολουθία, αυξάνεται ο αριθμός συνιστώσας για τις επόμενες κορυφές.

Ο αλγόριθμος είναι βέλτιστος, αφού είναι γραμμικός ως προς το μέγεθος της εισόδου του - $O(N + M)$, όπου N το πλήθος των κορυφών και M το πλήθος των ακμών.

ΑΛΓΟΡΙΘΜΟΣ 4.2: *Two Edge Connected Components - Bridge Tree*

```

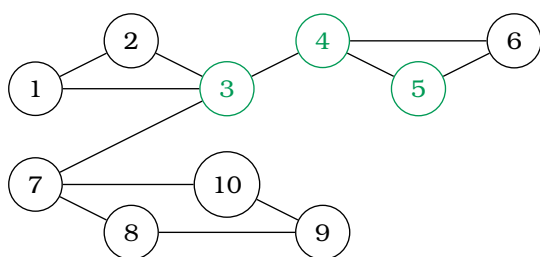
1: function COMPUTE_BRIDGE_TREE(graphG)
2:    $n \leftarrow$  vertices size of the graph
3:   unique_component_number  $\leftarrow 0$ 
4:   Initialize visited arrays, all set to false
5:   //mark the bridges in the graphG
6:   COMPUTE_BRIDGES(graphG)
7:   for each vertex  $i = 1, 2, 3, \dots, n$  do
8:     if visited[ $i$ ] = false then
9:       BRIDGE_TREE_UTIL( $i$ , unique_component_number, visited, graphG)
10:      unique_component_number  $\leftarrow$  unique_component_number + 1
11:    end if
12:  end for
13: end function
14:
15: function BRIDGE_TREE_UTIL(vertex, visited, component_number, graphG)
16:   component[vertex]  $\leftarrow$  component_number
17:   visited[vertex]  $\leftarrow$  true
18:   for each next adjacent to vertex do
19:     if edge(next, vertex) is not a bridge and visited[next] = false then
20:       BRIDGE_TREE_UTIL(next, visited, component_number, graphG)
21:     end if
22:   end for
23: end function

```

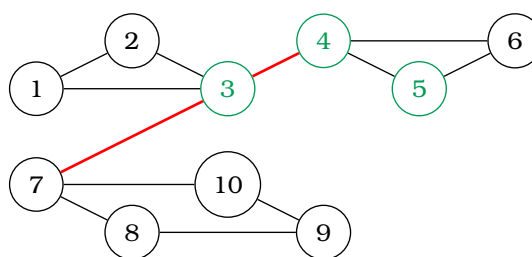
4.2.3 Ακμικές Δισυνεκτικές Συνιστώσες με Ενεργές Κορυφές

Μια παραλλαγή του προβλήματος αφορά κορυφές που είναι ενεργές. Σε αυτή την περίπτωση, μια ακμική δισυνεκτική συνιστώσα χαρακτηρίζεται ως ενεργή αν περιλαμβάνει τουλάχιστον μία ενεργή κορυφή.

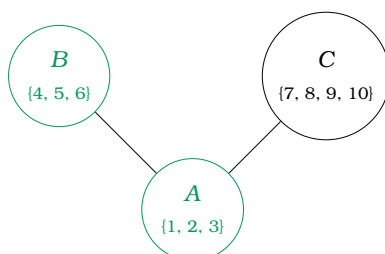
Ο αλγόριθμος είναι μια επέκταση του προηγούμενου ελέγχοντας για την ύπαρξη ενεργών κορυφών κατά την κατασκευή των συνιστωσών. Ο αλγόριθμος επιστρέφει ένα δεντρικό γράφημα (bridge tree) με αντιπροσώπους κορυφών (component representatives) για κάθε ακμική δισυνεκτική συνιστώσα με τις γέφυρες ως ακμές και έναν πίνακα αντιπροσώπων (bridge tree component map) για την αντιστοίχιση.



Σχήμα 4.2: Παράδειγμα αρχικού γραφήματος. Με πράσινο φαίνονται οι ενεργές κορυφές του



Σχήμα 4.3: Εύρεση των γεφυρών του γραφήματος



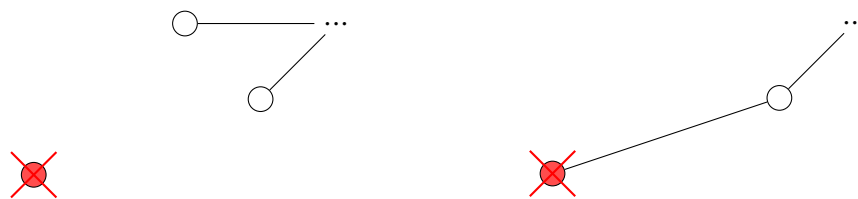
Σχήμα 4.4: Παραγόμενο δεντρικό ισοδύναμο γράφημα από το αρχικό

4.2.4 “Κλάδεμα” Δεντρικού Γραφήματος

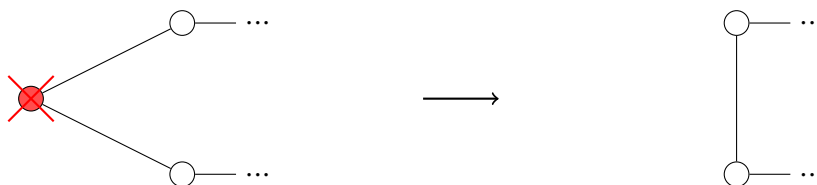
Η διαδικασία της σύντμησης, ή αλλιώς του *κλαδέματος* (pruning), εφαρμόζεται στο παραγόμενο δεντρικό γράφημα με σκοπό τη μείωση του μεγέθους του. Αν μία κορυφή είναι ανενεργή και έχει βαθμό ≤ 2 , τότε:

1. Αν ο βαθμός είναι 0 ή 1, αφαιρείται
2. Αν ο βαθμός είναι 2, οι γείτονές της ενώνονται με ακμή (edge contraction) και η ίδια αφαιρείται.

Ο αλγόριθμος είναι βέλτιστος, αφού και αυτός είναι γραμμικός ως προς το μέγεθος της εισόδου του, δηλαδή $O(N + M)$, όπου N το πλήθος των κορυφών και M το πλήθος των ακμών



Σχήμα 4.5: Παράδειγμα αφαίρεσης ανενεργής κορυφής βαθμού 0 και 1



Σχήμα 4.6: Παράδειγμα αφαίρεσης ανενεργής κορυφής βαθμού 2

του δεντρικού γραφήματος. Επιστρέφει, επίσης, το νέο κλαδεμένο δεντρικό γράφημα και έναν νέο πίνακα αντιπροσώπων (pruning component map) με μέγεθος της τάξης $O(l)$, όπου l το πλήθος των ενεργών κορυφών, που αντιπροσωπεύει τη πληροφορία της ακολουθίας των λειτουργιών που δίνεται μαζί με το γράφημα.

Παρακάτω παρέχεται ο αλγόριθμος σύντμησης σε ψευδοκώδικα:

ΑΛΓΟΡΙΘΜΟΣ 4.3: *Main Prune Function*

```

1: function PRUNE(graph, activeNodes, oldMap, newMap)
2:    $n \leftarrow \text{graph.total\_bridge\_vertices}()$ 
3:   Initialize removed array of size  $n$  with false
4:   PRUNE_INACTIVE_WITH_DEGREE_1(graph, removed)
5:   PRUNE_INACTIVE_WITH_DEGREE_2(graph, removed)
6:   PRUNE_INACTIVE_WITH_DEGREE_0(graph, removed)
7:   Initialize prunedMap array of size  $n$ 
8:   UPDATE_GRAPH(graph, prunedMap, removed)
9:   for all  $u$  in activeNodes do
10:     $\text{newMap}[u] \leftarrow \text{prunedMap}[\text{graph.bridgetreeMap}[\text{oldMap}[u]]]$ 
11:   end for
12: end function
13: function UPDATE_GRAPH(graph, prunedMap, removed)
14:    $\text{newId} \leftarrow 0$ 
15:   for  $v = 0$  to  $\text{graph.total\_bridge\_vertices}() - 1$  do
16:     if  $\text{removed}[v] = \text{false}$  then
17:        $\text{prunedMap}[v] \leftarrow \text{newId}$ 
18:        $\text{newId} \leftarrow \text{newId} + 1$ 
19:     else
20:        $\text{prunedMap}[v] \leftarrow -1$ 
21:     end if
22:   end for
23:   Initialize newGraph with  $\text{newId}$  vertices
24:   for  $v = 0$  to  $\text{graph.total\_bridge\_vertices}() - 1$  do
25:     if  $\text{removed}[v] = \text{false}$  then
26:       for all  $u$  in  $\text{graph.neighbors}(v)$  do
27:         if  $\text{removed}[u] = \text{false}$  then
28:            $\text{newGraph.add\_edge}(\text{prunedMap}[v], \text{prunedMap}[u])$ 
29:         end if
30:       end for
31:     end if
32:   end for
33:    $\text{graph.replace\_with}(\text{newGraph})$ 
34: end function

```

ΑΛΓΟΡΙΘΜΟΣ 4.4: *Prune Inactive Vertices by Degree*

```

1: function PRUNE_INACTIVE_WITH_DEGREE_1(graph, removed)
2:   leaves  $\leftarrow$  graph.get_leaves()
3:   while leaves not empty do
4:     leaf  $\leftarrow$  leaves.dequeue()
5:     if leaf is inactive then
6:       for all neighbor in graph.neighbors(leaf) do
7:         if neighbor not removed then
8:           parent  $\leftarrow$  neighbor
9:           parent.degree  $\leftarrow$  parent.degree - 1
10:          if parent.degree = 1 and parent is inactive then
11:            leaves.enqueue(parent)
12:          end if
13:          break
14:        end if
15:      end for
16:      PRUNING(graph, leaf, removed)
17:    end if
18:  end while
19: end function
20:
21: function PRUNE_INACTIVE_WITH_DEGREE_2(graph, removed)
22:   for v = 0 to graph.total_bridge_vertices() - 1 do
23:     if v is inactive and v.degree = 2 then
24:       neighbors  $\leftarrow$  unremoved neighbors of v
25:       PRUNING(graph, v, removed)
26:       graph.add_edge(neighbors[0], neighbors[1])
27:     end if
28:   end for
29: end function
30:
31: function PRUNE_INACTIVE_WITH_DEGREE_0(graph, removed)
32:   for v = 0 to graph.total_bridge_vertices() - 1 do
33:     if v is inactive and v.degree = 0 then
34:       PRUNING(graph, v, removed)
35:     end if
36:   end for
37: end function
38: function PRUNING(graph, vertex, removed)
39:   removed[vertex]  $\leftarrow$  true
40:   graph.clear_edges(vertex)
41:   vertex.degree  $\leftarrow$  vertex.degree - 1
42: end function

```

4.2.5 Ανανέωση του Πίνακα Αντιπροσώπων

Αφού εκτελεστεί ο αλγόριθμος, χρειάζεται ανανέωση του πίνακα αντιπροσώπων (component map update) για τις ενεργές αρχικές κορυφές:

ΑΛΓΟΡΙΘΜΟΣ 4.5: *update components (based only on initial active vertices)*

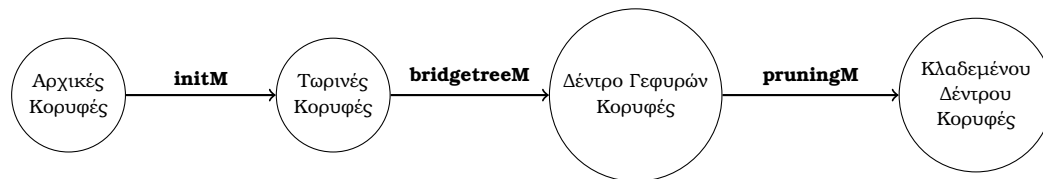
```

1: for each vertex  $v \in V_{\text{active}}^{\text{original}}$  do
2:    $\text{initM}[v] \leftarrow \text{pruningM}[\text{bridgeTreeM}[\text{initM}[v]]]$ 
3: end for

```

όπου:

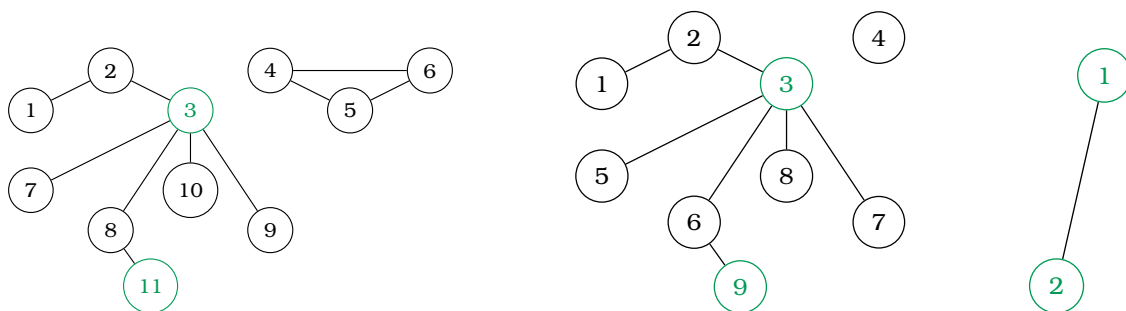
- $V_{\text{active}}^{\text{original}}$: το σύνολο των ενεργών κορυφών, στο αρχικό γράφημα
- initM : πίνακας αντιστοίχισης των αρχικών κορυφών με κορυφές του τρέχοντος γραφήματος.
- bridgeTreeM : πίνακας αντιστοίχισης των συνιστωσών μετά την κατασκευή του δέντρου γεφυρών με τις τρέχουσες κορυφές.
- pruningM : πίνακας αντιστοίχισης των νέων κορυφών μετά το κλάδεμα με τις κορυφές από την αρχική κατασκευή του δεντρικού γραφήματος.



Σχήμα 4.7: Αναπαράσταση των σχέσεων των διαδικασιών μεταξύ των επεξεργασιών ενός γραφήματος

4.2.6 Ισοδύναμο Γράφημα

Η διαδικασία δημιουργίας του ισοδύναμου γραφήματος από το αρχικό γράφημα περιγράφεται σε τέσσερα στάδια, ως σύνθεση των υπορουτινών που περιγράφηκαν προηγουμένως. Ο αλγόριθμος ακολουθεί τα εξής βήματα:



Σχήμα 4.8: Παράδειγμα από το αρχικό στο ισοδύναμο γράφημα

Αρχικές	Ενδιάμεσες	Τελικές
3	3	1
11	9	2

Σχήμα 4.9: Πίνακας Αντιπροσώπων Κορυφών Κληδεμένου/Ισοδύναμου Γραφήματος

1. Εντοπισμός και Μαρκάρισμα Γεφυρών
2. Κατασκευή του Δεντρικού Γραφήματος
3. “Κλάδεμα” Δεντρικού Γραφήματος - Δημιουργία Ισοδύναμου Γραφήματος
4. Ανανέωση του πίνακα των αντιπροσώπων

Οπότε, στο τέλος παράγεται ένα αραιομένο γράφημα που καλείται *ισοδύναμο* και συνοδεύεται από έναν ανανεωμένο πίνακα αντιπροσώπων, δεδομένης μιας ακολουθίας λειτουργιών. Στο παράδειγμα του Σχήματος 4.8 παρατηρείται ότι η τελική ισοδύναμη μορφή του γραφήματος είναι πολύ πιο απλή σε σχέση με το αρχικό, εμπεριέχοντας μόνο 2 κορυφές και ένα πίνακα αντιπροσώπων μόνο 2 αρχικών κορυφών (βλ. Σχήμα 4.9), των 3 και 11, όπου αντιστοιχούν στις νέες τρέχοντες κορυφές αντιπροσώπους 1 και 2, αντίστοιχα.

Τέλος, ο αλγόριθμος αυτός καταφέρνει να είναι βέλτιστος, αφού τρέχει σε γραμμικό τρόπο ως προς το μέγεθος του τρέχοντος γραφήματος ($O(N + M)$) και φτιάχνει ένα γράφημα συνοδευόμενο με έναν πίνακα αντιπροσώπων μεγέθους συνολικού μεγέθους $O(l)$, όπου l είναι το πλήθος των ενεργών κορυφών.

ΑΛΓΟΡΙΘΜΟΣ 4.6: *compute the equivalent graph*

```
1: input:
2:   graph - The initial graph.
3:   original_active_vertices - List of original active vertices.
4:   current_active_vertices - List of current active vertices.
5:   component_map - Mapping of components.
6: output:
7:   pruned-graph - The sparse equivalent graph.
8:   component_map - Updated component map.

9: procedure COMPUTE_EQUIVALENT_GRAPH
10:  function BRIDGES_STAGE
11:    Find all bridges in graph
12:  end function
13:  function 2-EDGE_CONNECTED_COMPONENTS_STAGE
14:    Construct the bridge – tree – graph from the bridges
15:    for each vertex original_vertex in graph do
16:      bridge_tree_mapping[original_vertex] = new_vertex
17:    end for
18:    Find bridge_tree – active_vertices
19:  end function
20:  function PRUNING_STAGE
21:    Construct the pruned – graph from bridge-tree-graph and
22:    bridge_tree-active_vertices
23:    for each vertex v in bridge-tree-graph do
24:      pruning_mapping[v] = pruned_vertex
25:    end for
26:  end function
27:  function UPDATE_COMPONENTS_STAGE
28:    Update component_map
29:    for each vertex v in original_active_vertices do
30:      component_map[v] =
31:        pruning_mapping[bridge_tree_mapping[component_map[v]]]
32:    end for
33:  end function
34:  return pruned-graph, component_map
35: end procedure
```

Ανάλυση και Σχεδίαση

Στο κεφάλαιο αυτό παρουσιάζεται η μελέτη που πραγματοποιήθηκε για την υλοποίηση, με κύρια έμφαση στον προτεινόμενο αλγόριθμο επίλυσης του προβλήματος της παρούσας εργασίας. Αρχικά, περιγράφεται η αρχιτεκτονική του αλγορίθμου, ενώ στη συνέχεια παρατίθεται η περιγραφή του σε ψευδοκώδικα. Ακολουθώντας, παρουσιάζεται και ένας απλοϊκός αλγόριθμος με σκοπό τη συγκριτική πειραματική αξιολόγηση, προκειμένου να ελεγχθεί η εγκυρότητα και η αποδοτικότητα του προτεινόμενου αλγορίθμου.

5.1 Βέλτιστο Αλγοριθμικό Μοντέλο

5.1.1 Δεδομένα Εισόδου

Μετά την ανάλυση του προηγούμενου κεφαλαίου, αναφέρθηκε ότι για την αποδοτικότερη εκτέλεση του αλγορίθμου χρειάζεται να επιτευχθεί πριν από αυτό μια προεπεξεργασία της ακολουθίας λειτουργιών. Με βάση αυτό, τα δεδομένα εισόδου για το αλγοριθμικό μοντέλο είναι τα εξής (βλ. Πίνακα 4.3):

- κενό γράφημα $G(V, E = \emptyset)$,
- ταυτοτικός πίνακας αντιπροσώπων $comp : \forall v \in V : comp[v] := v$,
- επαυξημένη ακολουθία λειτουργιών όπου περιέχει στην αρχή και τις αρχικές ακμές του γραφήματος ως *εισαγωγές ακμών*, με τα γνωστά στοιχεία:
 1. $I(i, j, D_{index} || \infty)$: εισαγωγής ακμής και το τότε έχει γίνει η διαγραφή της,
 2. $D(i, j, I_{index} || - \infty)$: διαγραφής ακμής και το τότε έχει γίνει η εισαγωγή της,
 3. $Q(i, j, -)$: ερωτήματα ακμικής δισυνεκτικότητας

5.1.2 Αρχιτεκτονική Αλγορίθμου

Δεδομένης μιας τέτοιας εισόδου, το πρώτο βήμα του αναδρομικού προτεινόμενου αλγορίθμου με λογική *διαίρει & βασίλευε* είναι αρχικά να χωρίσει το διάστημα λειτουργιών στη μέση και να έχει ένα γράφημα G_h , ένα πίνακα αντιπροσώπων $comp_h$ και μία ακολουθία λειτουργιών $events_h$, για κάθε μισό. Έτσι, σε κάθε μέρος γίνονται τα εξής:

1. Βρίσκει τις αρχικές ενεργές κορυφές στο $events_h$ και τις αντίστοιχες ενεργές κορυφές στο τρέχον γράφημα μέσω του $comp_h$,
2. Βρίσκει τις αρχικές μόνιμες ακμές μέσω του $events_h$ κι έπειτα μέσω του $comp_h$ τις μόνιμες ακμές του τρέχοντος γραφήματος και τις προσθέτει G_h ,

3. Υπολογίζει το ισοδύναμο γράφημα G_h'' από το G_h και το καινούργιο ανανεωμένο $comp_h''$ από το $comp_h$,
4. καλείται η διαδικασία αυτή πάλι αναδρομικά με εισόδο τώρα :
 - $events_h$: τις μισές λειτουργίες,
 - G_h'' : το ισοδύναμο γράφημα και
 - $comp_h''$: τον ανανεωμένο πίνακα αντιπροσώπων

έως ότου το μέγεθος του διαστήματος των λειτουργιών να γίνει τετριμμένο, δηλαδή $|events_h| = 1$. Τότε, αν η λειτουργία αυτή είναι ερώτημα $Q(i, j, -)$ η απάντηση βρίσκεται στον ανανεωμένο πίνακα αντιπροσώπων $comp_h$, με έναν απλό έλεγχο: αν οι δύο κορυφές έχουν τον ίδιο αντιπρόσωπο, ή αλλιώς, βρίσκονται στην ίδια ακμική δισυνεκτική συνιστώσα:

$$απάντηση := comp[i] == comp[j]$$

Έτσι, ο αλγόριθμος απαντά έγκυρα και βέλτιστα σε χρόνο εκτέλεσης $O(t \cdot \log t)$, που συνεπάγεται με το βέλτιστο $O(t \cdot \log n)$, για πολυωνυμικά φραγμένο t (βλ. 3.1.4).

ΑΛΓΟΡΙΘΜΟΣ 5.1: *Two Edge Connectivity*

```

1: procedure TWO_EDGE_CONNECTIVITY(operations, graph)
2:   events:
3:     compute the augmented operations (graph now is empty)
4:   results:
5:     initialize an empty list for the queries' results
6:   comp:
7:     define an empty array
8:   // initialize the components
9:   for each vertex  $v$  in graph do
10:     $comp[v] = v$ 
11:   end for
12:   call compute_two_edge_connectivity (
13:     events,
14:     graph,
15:     comp,
16:     results
17:   )
18: end procedure

```

ΑΛΓΟΡΙΘΜΟΣ 5.2: *compute two edge connectivity*

```

1: function COMPUTE_TWO_EDGE_CONNECTIVITY(operations, graph, component_map, re-
   results)
2:   if |operations| = 1 and operation's type = query then
3:     if component_map[u] = component_map[v] then
4:       res ← true
5:     else
6:       res ← false
7:     end if
8:     Add res to results
9:   else
10:    // split operations into two halves
11:    for each half do
12:      half_graph:
13:        a copy of the graph
14:      half_component_map:
15:        a copy of the component_map
16:      half_operations:
17:        compute half operations
18:      half_active_vertices:
19:        compute active vertices for the half_operations
20:      half_component_active_vertices:
21:        compute component active vertices from half_active_vertices
22:        through component_map
23:      half_permanent_edges:
24:        compute permanent edges
25:      half_component_permanent_edges:
26:        compute component permanent edges from half_permanent_edges
27:        through component_map
28:      add half_component_permanent_edges to graph
29:      call compute_equivalent_graph(
30:        half_graph,
31:        half_active_vertices,
32:        half_component_active_vertices,
33:        half_component_map
34:      )
35:      call compute_two_edge_connectivity (
36:        half_operations,
37:        half_graph,
38:        half_component_map
39:      )
40:    end for
41:  end if
42: end function

```

5.2 Απλός Αλγόριθμος Αναφοράς

Ο αλγόριθμος με τον οποίο θα συγκριθεί ο προτεινόμενος, ο οποίος αξιολογείται σε αυτή την εργασία, έχει χρονική πολυπλοκότητα $O(t \cdot (N + M))$. Παρόμοια, ως είσοδο δέχεται ένα αρχικό γράφημα και μια ακολουθία λειτουργιών, την οποία ξεκινά και την εκτελεί σειριακά όπως αυτή του δίνεται, χωρίς καμιά αλλαγή ή προεπεξεργασία. Σε κάθε λειτουργία εκτελεί τα εξής πολυπλοκότητα:

- $\text{insert}(u, v)$: Εκτελεί αφαίρεση ακμής του γραφήματος. Σε μια απλή αναπαράσταση του γραφήματος αυτό γίνεται σε σταθερό χρόνο $O(1)$
- $\text{delete}(u, v)$: Εκτελεί διαγραφή ακμής, ψάχνοντας στην αναπαράσταση του γραφήματος την ακμή $O(\min\{N, M\})$
- $\text{query}(u, v)$: Εκτελεί την εύρεση του δεντρικού ισοδύναμου γραφήματος, στην πρώτη του φάση, όπως περιγράφηκε προηγουμένως, το οποίο κάνει συνολικό χρόνο $O(N + M)$. Η απάντηση στο ερώτημα των δύο κορυφών είναι άμεση κοιτώντας ποια είναι η δισυνεκτική συνιστώσα τους

Η επιλογή του αλγορίθμου αυτού εξυπηρετεί διπλό σκοπό: αφενός, αποτελεί μέτρο σύγκρισης για την αποδοτικότητα του υπό εξέταση αλγορίθμου, και αφετέρου λειτουργεί ως μέθοδος επιβεβαίωσης της ορθότητας των παραγόμενων αποτελεσμάτων.

ΑΛΓΟΡΙΘΜΟΣ 5.3: *Simple Reference Algorithm*

```

1: Input: Initial graph:  $G$ , operations sequence:  $L$ 
2: Output: Query results
3: Initialize: An empty list for the query results:  $results$ 
4: for each operation in  $L$  do
5:   if operation =  $\text{insert}(u, v)$  then
6:     Insert edge  $(u, v)$  into  $G$ 
7:   else if operation =  $\text{delete}(u, v)$  then
8:     Delete edge  $(u, v)$  from  $G$ 
9:   else if operation =  $\text{query}(u, v)$  then
10:    Find bridges in  $G$ 
11:    Construct the bridge tree
12:    if  $(u, v)$  are in the same connected component in the bridge tree then
13:       $res \leftarrow \text{TRUE}$ 
14:    else
15:       $res \leftarrow \text{FALSE}$ 
16:    end if
17:    Add  $res$  to  $results$ 
18:  end if
19: end for
20: return  $results$ 

```

Part

Πειραματικό Μέρος

Πειράματα & Αποτελέσματα

Στο κεφάλαιο αυτό περιγράφονται τα πειράματα που διεξήχθησαν με τους δύο προαναφερθείσες αλγορίθμους στο προηγούμενο κεφάλαιο. Στόχος της πειραματικής διαδικασίας είναι η διερεύνηση της αποδοτικότητας του προτεινόμενου αλγορίθμου συγκρινόμενο με αυτή του αλγορίθμου αναφοράς.

6.1 Παραγωγή Εισόδου

Για την αξιολόγηση των αλγορίθμων, δημιουργήθηκαν αρχικά γραφήματα και αντίστοιχες ακολουθίες λειτουργιών, με στόχο τη σύγκριση τόσο της αποδοτικότητας όσο και της ορθότητας των αποτελεσμάτων.

Η διαδικασία κατασκευής της εισόδου περιλάμβανε τα εξής:

- **Αρχικά (Πυκνά) Γραφήματα:** Δημιουργήθηκαν 7 διαφορετικά απλά γραφήματα χωρίς βρόχους, με αυξανόμενο πλήθος κορυφών και ακμών μέχρι τάξης του 10^4 . Τα γραφήματα ικανοποιούν τη συνθήκη $N < M < \frac{N^2}{2}$ ώστε να εξασφαλίζεται η πυκνότητά τους.
- **Λειτουργίες:** Για κάθε γράφημα παράχθηκαν 5 διαφορετικές ακολουθίες λειτουργιών, των οποίων το μέγεθος αυξάνεται γραμμικά. Συγκεκριμένα, το i -οστή ακολουθία λειτουργιών σε ένα αρχικό γράφημα $G(V, E)$ έχει μήκος $i \times N$, όπου N το πλήθος των κορυφών V του γραφήματος. Οι ακολουθίες λειτουργιών κατασκευάστηκαν ώστε να περιλαμβάνουν κατά το ήμισυ ερωτήματα σχετικά με τη συνδεσιμότητα του γραφήματος και κατά το υπόλοιπο ήμισυ μεταβολές στη δομή του, όπως εισαγωγές ή διαγραφές ακμών, ώστε να πετυχαίνεται πυκνότητα ως προς τα ερωτήματα ακμικής δισυνεκτικότητας.
- **Συνδυασμός Δεδομένων:** Κάθε πείραμα αποτελείται από ένα γράφημα και τις 5 αντίστοιχες ακολουθίες λειτουργιών του, επιτρέποντας συνεπή και συγκρίσιμη αξιολόγηση μεταξύ των δύο αλγορίθμων.

Η εν λόγω κατασκευή επιτρέπει την εμπεριστατωμένη πειραματική μελέτη, υπό ελεγχόμενες και κλιμακούμενες συνθήκες, των δυνατοτήτων κάθε αλγορίθμου τόσο ως προς την ταχύτητα όσο και ως προς την ακρίβεια των απαντήσεών του.

6.2 Διαγράμματα Αποτελεσμάτων

Στόχος του πειραματικού μέρους είναι η επιβεβαίωση της θεωρητικής ανάλυσης ως προς την υπολογιστική πολυπλοκότητα των δύο αλγορίθμων. Συγκεκριμένα, αναμένεται ότι καθώς αυξάνεται το πλήθος των εκτελούμενων λειτουργιών, η διαφορά στην απόδοσή τους θα γίνεται εντονότερη, αντανakλώντας την ασυμπτωτική διαφορά τους, μεταξύ $O(t \cdot \log n)$ και $O(t \cdot (N + M))$, τόσο θεωρητική όσο και πρακτική (βλ. Παράρτημα Γ').

Στα επόμενα διαγράμματα παρουσιάζονται συγκριτικά οι χρόνοι εκτέλεσης των δύο αλγορίθμων για κάθε διαφορετική περίπτωση εισόδου. Η οπτικοποίηση συνοδεύεται από συγκεντρωτικούς πίνακες με τις αντίστοιχες μετρήσεις χρόνου σε δευτερόλεπτα (*seconds*), επιτρέποντας άμεση και ποσοτική σύγκριση της αποδοτικότητας των συγκριθέντων αλγορίθμων.

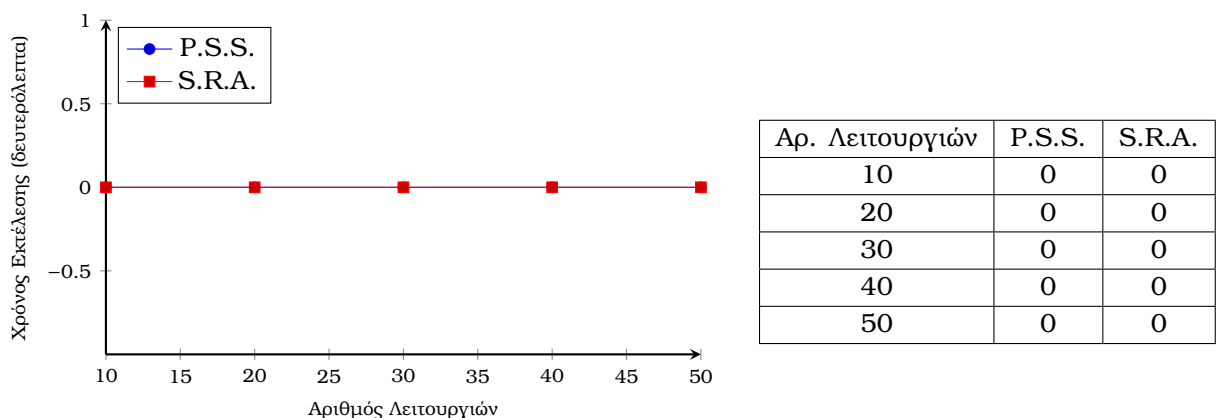
6.2.1 Περιπτώσεις Μικρής Κλίμακας ($N \in \{10, 40, 100\}$)

Αρχικά, στα πρώτα πειραματικά αποτελέσματα που αντιστοιχούν σε μικρού μεγέθους γραφήματα και ακολουθίες λειτουργιών ($N \in \{10, 40, 100\}$), παρατηρείται συγκλίνουσα συμπεριφορά μεταξύ του προτεινόμενου αλγόριθμου **P.S.S.** (Peng, Sandlund & Sleator) και του αλγόριθμου αναφοράς **S.R.A.** (Simple Reference Algorithm).

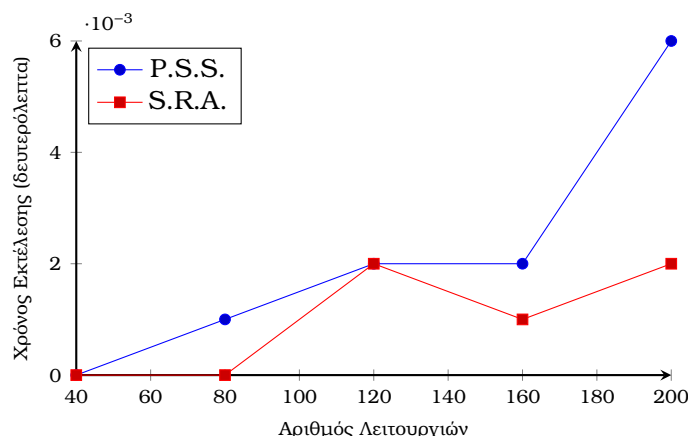
Η παρατήρηση αυτή ενισχύεται τόσο εμπειρικά όσο και θεωρητικά: όταν το μέγεθος του γραφήματος είναι μικρό ($|V| := N \approx O(1)$), οι χρονικές πολυπλοκότητες των δύο προσεγγίσεων συγκλίνουν, καθώς ισχύει $O(t \cdot \log n) = O(t) = O(t \cdot (N + M))$.

Επιπλέον, καταγράφεται ότι στις περιπτώσεις αυτές ο αλγόριθμος αναφοράς εμφανίζει ελαφρώς καλύτερους χρόνους εκτέλεσης συγκριτικά με τον προτεινόμενο (βλ. Σχήμα 6.2). Το φαινόμενο αυτό αποδίδεται πιθανόν στην απλότητα της υλοποίησής του. Αντίθετα, ο προτεινόμενος αλγόριθμος βασίζεται σε πιο περίπλοκες αναδρομικές διαδικασίες και επιπρόσθετο λογισμικό φορτίο, το οποίο ενδέχεται να επιβαρύνει τον χρόνο εκτέλεσης σε περιπτώσεις μικρής κλίμακας.

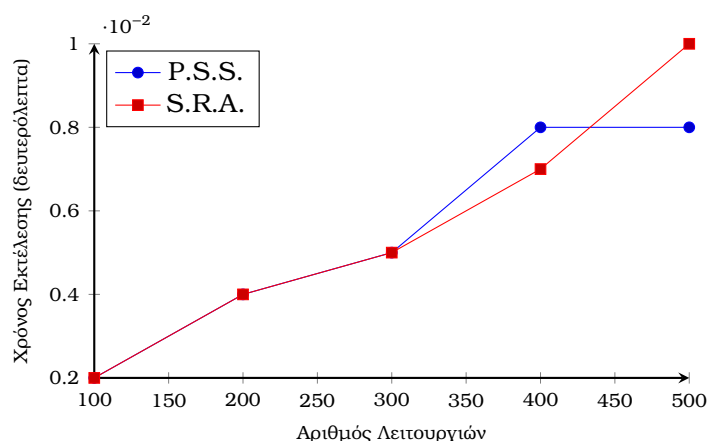
Παρακάτω παρέχονται τα πειραματικά αποτελέσματα σε περιπτώσεις μικρής κλίμακας:



Σχήμα 6.1: Γράφημα 1: $G(V = 10, E = 19)$



Σχήμα 6.2: Γράφημα 2: $G(V = 40, E = 71)$

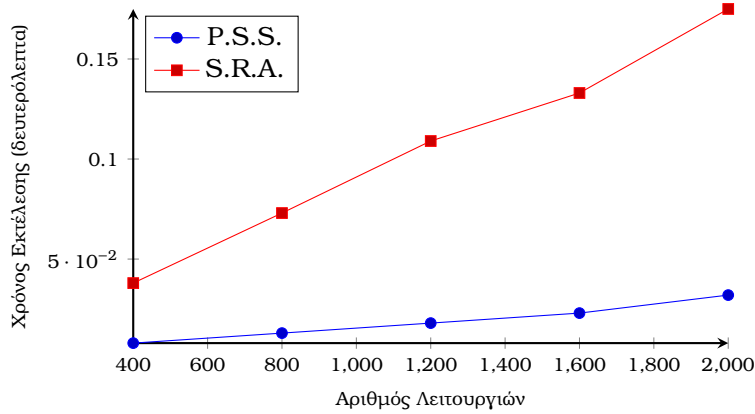
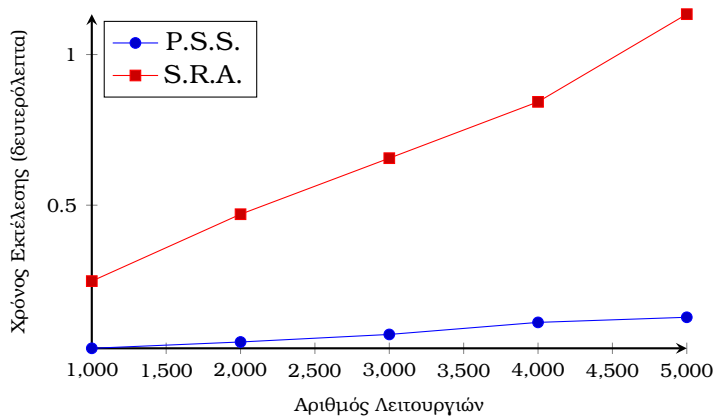


Σχήμα 6.3: Γράφημα 3: $G(V = 100, E = 175)$

6.2.2 Περιπτώσεις Μεσαίας Κλίμακας ($N \in \{400, 1000, 3000\}$)

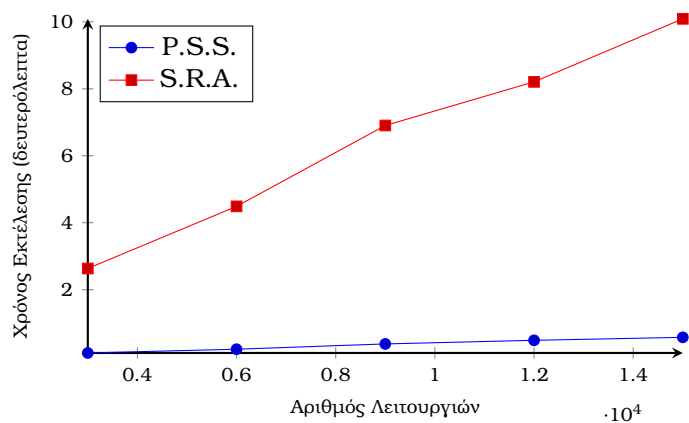
Σε αυτή τη φάση της αξιολόγησης, παρατηρείται πλέον σαφής διαφοροποίηση στον χρόνο εκτέλεσης των δύο αλγορίθμων καθώς αυξάνεται το μέγεθος της εισόδου. Ήδη από το τελευταίο διάγραμμα της παρούσας ενότητας, είναι εμφανές ότι ο αλγόριθμος αναφοράς αρχίζει να εμφανίζει σημαντική χρονική υστέρηση, φτάνοντας στα όρια της πρακτικής εφαρμοσιμότητάς του.

Αντιθέτως, ο προτεινόμενος αλγόριθμος εξακολουθεί να παρουσιάζει σταθερή και προβλέψιμη συμπεριφορά, ανταποκρινόμενος ικανοποιητικά στις αυξημένες απαιτήσεις των συγκεκριμένων παραδειγμάτων. Παρακάτω παρέχονται τα πειραματικά αποτελέσματα σε περιπτώσεις μεσαίας κλίμακας:

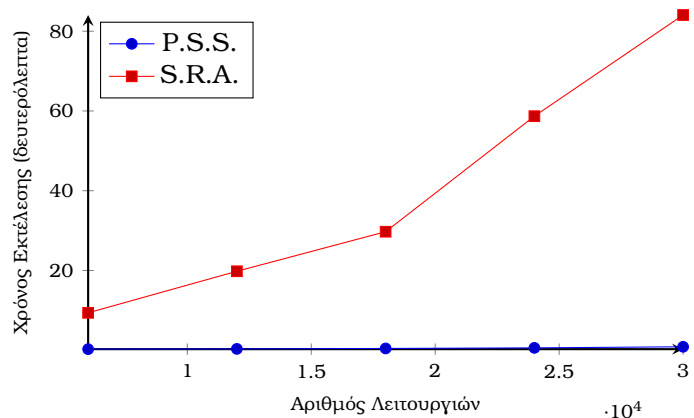
Σχήμα 6.4: Γράφημα 4: $G(V = 400, E = 811)$ Σχήμα 6.5: Γράφημα 5: $G(V = 1000, E = 1479)$

6.2.3 Περιπτώσεις Μεγάλης Κλίμακας ($N \in \{6000, 10000\}$)

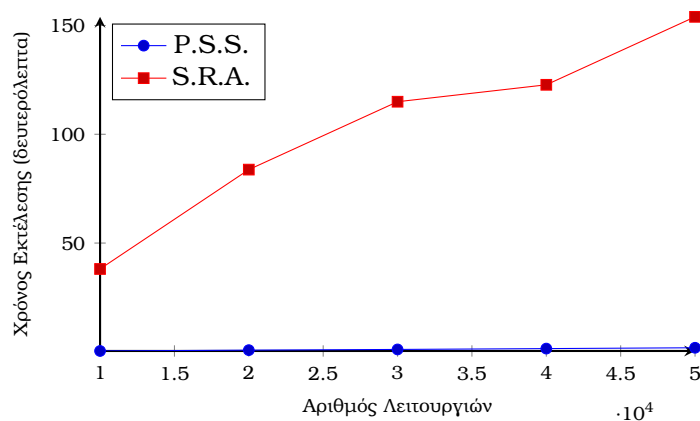
Στη τελική αυτή φάση που αρχίζει να αντικατοπτρίζει το μέγεθος δεδομένων πραγματικών προβλημάτων, ο αλγόριθμος αναφοράς φαίνεται ότι έχει σταματήσει να είναι επιλογή χρήσης, ενώ ακόμη ο προτεινόμενος αλγόριθμος *P.S.S.* στέκεται αρκετά καλά υπό αυτές τις εισόδους, επιβεβαιώνοντας έτσι την θεωρητική του πολυπλοκότητα και την εργασία των δημιουργών του. Παρακάτω παρέχονται τα πειραματικά αποτελέσματα σε περιπτώσεις πραγματικής κλίμακας:



Σχήμα 6.6: Γράφημα 6: $G(V = 3000, E = 3034)$



Σχήμα 6.7: Γράφημα 7: $G(V = 6000, E = 9819)$



Σχήμα 6.8: Γράφημα 8: $G(V = 10000, E = 10482)$

6.3 Συμπεράσματα

Τα πειραματικά αποτελέσματα επιβεβαιώνουν πλήρως τις θεωρητικές προσδοκίες. Σε περιπτώσεις μικρής εισόδου, ο αλγόριθμος αναφοράς εμφανίζει ελαφρώς καλύτερη ή ισοδύναμη επίδοση σε σχέση με το προτεινόμενο μοντέλο, λόγω της απλότητας των λειτουργιών και του περιορισμένου υπολογιστικού φορτίου. Ωστόσο, καθώς το πλήθος των λειτουργιών αυξάνεται, οι χρόνοι εκτέλεσης των δύο μεθόδων αποκλίνουν σημαντικά, με τον προτεινόμενο αλγόριθμο να υπερέχει κατά πολύ σε απόδοση.

Η ραγδαία αυτή απόκλιση επιβεβαιώνει αφενός την ορθότητα της θεωρητικής ανάλυσης κι αφ' ετέρου τη λειτουργική επάρκεια της υλοποίησης, συμφωνώντας και με την εμπειρική σχέση μεταξύ πολυπλοκότητας χρόνου και το μέγεθος της εισόδου (βλ. Πίνακα Γ.1). Η εν λόγω παρατήρηση ενισχύει τα συμπεράσματα της παρούσας μελέτης και συμφωνεί με τις θεωρητικές εργασίες της βιβλιογραφίας [2] στις οποίες βασίστηκε.

Μέρος

Επίλογος

Επίλογος

7.1 Μελλοντικές Επεκτάσεις

Ο αλγόριθμος του αλγοριθμικού θεωρητικού μοντέλου που αναπτύχθηκε στο πλαίσιο της παρούσας διπλωματικής εργασίας μπορεί να αποτελέσει τη βάση για περαιτέρω εξέλιξη και διεύρυνση σε πολλαπλά επίπεδα. Συγκεκριμένα, οι βασικές κατευθύνσεις μελλοντικής έρευνας και ανάπτυξης περιλαμβάνουν:

1. **Υποστήριξη γραφημάτων με πολλαπλές ακμές (multi-edge graphs):** Με σχετικά μικρές τροποποιήσεις στη φάση προεπεξεργασίας των λειτουργιών (operations preprocessing), η παρούσα υλοποίηση του αλγορίθμου μπορεί να αναπροσαρμοστεί ώστε να διαχειρίζεται γραφήματα με πολλαπλές ακμές, χωρίς να απαιτούνται αλλαγές στους βασικούς αλγοριθμικούς μηχανισμούς, αφού η βασική υπορουτίνα δημιουργίας του αραιομένου *ισοδύναμου* γραφήματος διαχειρίζεται ούτως ή άλλως αυτή την περίπτωση.
2. **Πλήρης ολοκλήρωση συμπεριλαμβανομένου t εκθετικά φραγμένου από το n :** Η παρούσα έκδοση του αλγορίθμου μπορεί να επεκταθεί και να ενισχυθεί με στόχο την υλοποίηση όλων των θεωρητικών παραλλαγών που αναλύθηκαν στο αντίστοιχο θεωρητικό μέρος.
3. **Διεύρυνση πειραματικού πλαισίου ελέγχου:** Προτείνεται η ενίσχυση της πειραματικής αξιολόγησης με τη χρήση πιο ποικίλων και μεγάλης κλίμακας συνόλων δεδομένων, καθώς και η ενσωμάτωση εντατικών πειραμάτων και ακραίων περιπτώσεων. Ένα πλήρες και ευέλικτο πλαίσιο ελέγχου θα συμβάλει σημαντικά στην αξιολόγηση της πρακτικής αξιοπιστίας και αποδοτικότητας της αλγορίθμου.
4. **Επέκταση σε πιο σύνθετα προβλήματα συνεκτικότητας:** Το αλγοριθμικό μοντέλο που στηρίχθηκε η υφιστάμενη υλοποίηση για το 2-ακμικά συνεκτικό πρόβλημα είναι αρκετά γενικότερο ώστε να μπορεί να επεκταθεί σε προβλήματα όπως:
 - 2-κομβική συνεκτικότητα
 - 3-ακμική/κομβική συνεκτικότητα
 - Πιθανή μελλοντική υποστήριξη για 4-ακμική/κομβική συνεκτικότητα

Το αλγοριθμικό μοντέλο λειτουργεί δηλαδή ως *σκελετός* επίλυσης για όλα τα παραπάνω προβλήματα. Αυτό υποδηλώνει ότι η μόνη διαφοροποίηση που χρειάζεται για

να χρησιμοποιηθεί η δομή του για την επίλυση κάποιου από τα παραπάνω συνεκτικά προβλήματα είναι μόνο ο αλγόριθμος υπολογισμού του *ισοδύναμου γραφήματος*, δηλαδή η αλλαγή της τεχνικής αрайοποίησης που χρησιμοποιείται σε κάθε αναδρομική κλήση για την επεξεργασία του τρέχοντος γραφήματος.

Παραρτήματα

Ορισμοί, Λήματα & Θεωρήματα

Σε αυτό το κεφάλαιο παρουσιάζονται τα βασικά λήματα και θεωρήματα που θεμελιώνουν τη θεωρητική πολυπλοκότητα του προτεινόμενου αλγορίθμου, ο οποίος αναλύθηκε και υλοποιήθηκε στο πλαίσιο της παρούσας εργασίας. Η θεωρητική τεκμηρίωση βασίζεται στο έργο των P.S.S. [2], όπου περιλαμβάνονται οι πλήρεις αποδείξεις τους. Για λόγους πληρότητας και κατανόησης, παρατίθεται εδώ η ουσιαστική μαθηματική επιχειρηματολογία επί της οποίας στηρίζεται η αποδοτικότητα της προσέγγισης, χωρίς τις αποδείξεις τους.

A'.1 Γενικά

Θεώρημα 1.1. Έστω $G = (V, E)$ μη κατευθυνόμενο γράφημα με $|V| = n$, στο οποίο εφαρμόζεται μία ακολουθία t λειτουργιών της μορφής:

- ΕΙΣΑΓΩΓΗ-ΑΚΜΗΣ(u, v),
- ΔΙΑΓΡΑΦΗ-ΑΚΜΗΣ(u, v),
- ΕΡΩΤΗΜΑ(u, v): αν οι u και v είναι 2-ακμικά ή κομβικά συνεκτικοί, 3-ακμικά ή κομβικά συνεκτικοί στο τρέχον γράφημα

Τότε, υπάρχει αλγόριθμος που εκτελεί όλες τις t λειτουργίες σε συνολικό χρόνο $O(t \cdot \log n)$.

Ορισμός 1.1. Έστω γράφημα $G = (V_G, E_G)$ με υποσύνολο κορυφών $W \subseteq V_G$ και γράφημα $H = (V_H, E_H)$ με $W \subseteq V_H$. Λέμε ότι οι G και H είναι **c-ακμικά ισοδύναμα**, εάν για κάθε διάσπαση (A, B) του W , το μέγεθος της ελάχιστης ακμής-τομής που διαχωρίζει το A από το B είναι το ίδιο στους G και H , όταν αυτό το μέγεθος είναι μικρότερο του c .

Ομοίως, λέμε ότι οι G και H είναι **c-κομβικά ισοδύναμα**, εάν για κάθε διάσπαση (A, B, C) του W με $|C| < c$, το μέγεθος της ελάχιστης κομβικής τομής D που διαχωρίζει το A από το B (όπου $C \subseteq D$, $D \cap A = \emptyset$, $D \cap B = \emptyset$) είναι το ίδιο στα G και H , όταν αυτό το μέγεθος είναι μικρότερο του c .

Λήμμα 1.1. Έστω ότι τα γραφήματα $G = (V_G, E_G)$ και $H = (V_H, E_H)$ είναι c -ακμικά ή c -κομβικά ισοδύναμα ως προς το σύνολο κορυφών W . Έστω E_W οποιοδήποτε σύνολο ακμών μεταξύ κορυφών του W . Τότε οι επεκτάσεις $G' = (V_G, E_G \cup E_W)$ και $H' = (V_H, E_H \cup E_W)$ παραμένουν c -ακμικά ή c -κομβικά ισοδύναμες, αντιστοίχως.

Λήμμα 1.2. Έστω γράφημα G με m ακμές και σύνολο κορυφών W μεγέθους k . Αν υπάρχει αλγόριθμος χρόνου $O(m)$ που κατασκευάζει γράφημα H μεγέθους $O(k)$ τέτοιο ώστε H και G να είναι c -ακμικά ή c -κομβικά ισοδύναμο ως προς το W , τότε υπάρχει αλγόριθμος που, για κάθε

ακολουθία λειτουργιών $x_1, \dots, x_t$ (εισαγωγές/διαγραφές ακμών και ερωτήματα συνεκτικότητας), απαντά σε όλα τα c -ακμικά ή c -κομβικά ερωτήματα συνεκτικότητας σε συνολικό χρόνο:

$$O(t \log n)$$

Α'.2 Ακμική Δισυνεκτικότητα

Λήμμα 1.3. Έστω S μια 2-ακμικά συνεκτική συνιστώσα ενός γραφήματος G . Αν συρρικνωθούν όλες οι κορυφές του S σε μία κορυφή s και αναπροσαρμοστούν ανάλογα τα άκρα των ακμών, προκύπτει γράφημα H που είναι 2-ακμικά ισοδύναμο με το G .

Λήμμα 1.4. Αν το γράφημα G είναι δέντρο, τότε οι παρακάτω δύο κανόνες παράγουν γράφημα H 2-ακμικά ισοδύναμο με το G :

- Αφαίρεση μη ενεργού φύλλου (βαθμός 1).
- Αφαίρεση μη ενεργής κορυφής βαθμού 2 και προσθήκη ακμής μεταξύ των δύο γειτόνων της.

Λήμμα 1.5. Δεδομένου ενός γραφήματος G με m ακμές και k ενεργές κορυφές W , μπορεί να κατασκευαστεί ένα γράφημα H , 2-ακμικά ισοδύναμο με το G και μεγέθους $O(k)$, σε χρόνο $O(m)$.

Θεμελιώδες Κάτω Φράγμα για Δυναμική Συνεκτικότητα

Στο πρόβλημα της *δυναμικής συνεκτικότητας*, με ένα μη κατευθυνόμενο γράφημα $G(V, E)$ που υφίσταται δυναμικές τροποποιήσεις (εισαγωγές και διαγραφές ακμών) απαντώνται συνεκτικά ερωτήματα:

“είναι οι κορυφές u και v συνεκτικές”.

Ένα από τα σημαντικότερα θεωρητικά αποτελέσματα στον τομέα είναι το *κάτω φράγμα* που απέδειξαν οι Patrascu & Demaine, το οποίο θέτει έναν θεμελιώδη περιορισμό στο πόσο αποδοτικά μπορούν να γίνουν τέτοιοι δυναμικοί αλγόριθμοι συνεκτικότητας.

Θεώρημα 2.2 (Κάτω Φράγμα - $\Omega(\log n)$ [11, 12]). *Κάθε αλγόριθμος που επιλύει το πρόβλημα της δυναμικής συνεκτικότητας, με πράξεις αλληλαγών και ερωτημάτων, απαιτεί χρόνο τουλάχιστον*

$$\Omega(\log n) - \text{ανά πράξη}$$

Το αποτέλεσμα αυτό ουσιαστικά δηλώνει ότι κανένας αλγόριθμος δεν μπορεί να απαντά σε ερωτήματα συνεκτικότητας γρηγορότερα από αυτό το κάτω φράγμα, αν θέλει ταυτόχρονα να υποστηρίζει δυναμικές τροποποιήσεις. Και αυτό αποδείχτηκε τόσο για το online όσο και για το offline πρόβλημα.

Πολυπλοκότητα Χρόνου σε Σχέση με το Μέγεθος Εισόδου

Η εκτίμηση της αποδοτικότητας ενός αλγορίθμου ως προς το μέγεθος εισόδου μπορεί να πραγματοποιηθεί βάσει της αναμενόμενης πολυπλοκότητας χρόνου. Ο ακόλουθος πίνακας παρουσιάζει τις πολυπλοκότητες χρόνου που θεωρούνται αποδεκτές υπό χρονικό περιορισμό 1 δευτερολέπτο, βασισμένος σε υπάρχουσα βιβλιογραφία [1, 18]:

Μέγεθος εισόδου n	Αποδεκτή πολυπλοκότητα χρόνου
$n \leq 10$	$O(n!)$
$n \leq 20$	$O(2^n)$
$n \leq 500$	$O(n^3)$
$n \leq 5\,000$	$O(n^2)$
$n \leq 10^6$	$O(n \cdot \log n)$ ή $O(n)$
Πολύ μεγάλο n	$O(1)$ ή $O(\log n)$

Πίνακας Γ.1: Σχέση πολυπλοκότητας χρόνου με το μέγεθος εισόδου [1]

Ενδεικτικά Παραδείγματα

- Για $n = 10^5$, ένας αλγόριθμος με πολυπλοκότητα $O(n^2)$ θα εκτελέσει περίπου 10^{10} πράξεις, αριθμός απαγορευτικός για χρονικό όριο ενός δευτερολέπτου.
- Στην περίπτωση πολυπλοκότητας $O(n \cdot \log n)$ και $n = 10^5$, το πλήθος των πράξεων εκτιμάται σε $10^5 \cdot \log_2(10^5) \approx 10^5 \cdot 17 \approx 1.7 \cdot 10^6$, ποσότητα που θεωρείται αποδεκτή.

Οι παραπάνω εκτιμήσεις συμβάλλουν στον έγκαιρο αποκλεισμό μη αποδοτικών στρατηγικών. Ενδεικτικά, για $n = 20$, η χρήση εξαντλητικής αναζήτησης με πολυπλοκότητα $O(2^n)$ δύναται να είναι αποδεκτή. Αντιθέτως, για $n = 10^6$, ενδείκνυνται μόνον αλγόριθμοι γραμμικής ή σχεδόν γραμμικής πολυπλοκότητας.

Επίδραση Σταθερών Παραγόντων

Η πολυπλοκότητα χρόνου παρέχει χρήσιμες εκτιμήσεις, χωρίς όμως να λαμβάνει υπόψη τους σταθερούς παράγοντες. Συγκεκριμένα:

- Για $n = 10^6$, ο Αλγόριθμος Α εκτελεί $100n$ πράξεις $\rightarrow 10^8$ συνολικές πράξεις
- Ο Αλγόριθμος Β εκτελεί $10n \log n$ πράξεις $\rightarrow$ περίπου $1.7 \cdot 10^7$ πράξεις

Παρά τη θεωρητικά χειρότερη ασυμπτωτική πολυπλοκότητα, ο δεύτερος αλγόριθμος ενδέχεται να είναι ταχύτερος στην πράξη λόγω μικρότερου συντελεστή.

Κατηγοριοποίηση Βιβλιογραφικών Αναφορών

Τύπος βιβλιογραφικής πηγής	Αναφορά
Άρθρο σε επιστημονικό περιοδικό	[3, 4, 5, 8, 10, 12, 21, 28]
Εισήγηση σε συνέδριο	[2, 11, 13, 24]
Τεχνική αναφορά / Επιστημονική αναφορά (misc)	[15, 16]
Ιστοσελίδα / Online υλικό (misc)	[26, 27, 29]
Βιβλία	[1, 17, 18]

Βιβλιογραφία

- [1] Antti Laaksonen. *Competitive Programmer's Handbook*. 2018. <https://cses.fi/book.html>.
- [2] Richard Peng, Bryce Sandlund και Daniel D. Sleator. *Optimal Offline Dynamic 2,3-Edge/Vertex Connectivity (v2)*. *Algorithms and Data Structures - 16th International Symposium, WADS 2019, Edmonton, AB, Canada, August 5-7, 2019, Proceedings*, τόμος 11632 στο *Lecture Notes in Computer Science*, σελίδες 602–616. Springer, 2019. <https://arxiv.org/abs/1708.03812v2>.
- [3] David Eppstein. *Offline algorithms for dynamic minimum spanning tree problems*. *Journal of Algorithms*, 17(2):237–250, 1994. <https://doi.org/10.1006/jagm.1994.1033>.
- [4] David Eppstein, Zvi Galil, Giuseppe F. Italiano και Thomas H. Spencer. *Separator Based Sparsification: I. Planarity Testing and Minimum Spanning Trees*. *J. Comput. Syst. Sci.*, 52(1):3–27, 1996. <https://sciencedirect.com/science/article/pii/S0022000096900096>.
- [5] David Eppstein, Zvi Galil, Giuseppe F. Italiano και Thomas H. Spencer. *Separator Based Sparsification II: Edge and Vertex Connectivity*. *SIAM J. Comput.*, 28(1):341–381, 1998. <https://epubs.siam.org/doi/10.1137/S0097539793244356>.
- [6] Monika Rauch Henzinger. *Fully dynamic biconnectivity in graphs*. *Algorithmica*, 13(6):503–538, 1995. <https://doi.org/10.1007/BF01189067>.
- [7] Monika Rauch Henzinger. *Improved data structures for fully dynamic biconnectivity*. *SIAM Journal on Computing*, 29(6):1761–1815, 2000. <https://doi.org/10.1137/S0097539794263907>.
- [8] Kathrin Hanauer, Monika Henzinger και Christian Schulz. *Recent Advances in Fully Dynamic Graph Algorithms*. *ACM Journal of Experimental Algorithmics*, 27(1):1.11:1–1.11:45, 2022. <https://arxiv.org/abs/2102.11169>.
- [9] David Eppstein, Zvi Galil, Giuseppe F. Italiano και Amnon Nissenzweig. *Sparsification—a technique for speeding up dynamic graph algorithms*. *Journal of the ACM (JACM)*, 44(5):669–696, 1997. <https://doi.org/10.1145/265910.265914>.
- [10] Jacob Holm, Kristiande Lichtenberg και Mikkel Thorup. *Poly-Logarithmic Deterministic Fully Dynamic Algorithms for Connectivity, Minimum Spanning Tree, 2-Edge, and Biconnectivity*. *SIAM J. Comput.*, 31(3):720–740, 2001. <https://dl.acm.org/doi/10.1145/502090.502095>.

- [11] Mihai Pătraşcu και Erik D. Demaine. *Lower Bounds for Dynamic Connectivity*. *Proceedings of the Thirty-Sixth Annual ACM Symposium on Theory of Computing (STOC '04)*, σελίδες 546–553. ACM, 2004. <https://doi.org/10.1145/1007352.1007435>.
- [12] Mihai Pătraşcu και Erik D. Demaine. *Logarithmic Lower Bounds in the Cell-Probe Model*. *SIAM Journal on Computing*, 35(4):932–963, 2006. doi:<https://doi.org/10.1137/S0097539705447256>, or as arXiv preprint:<https://arxiv.org/abs/cs/0502041>.
- [13] Jakub Karczmarz, Adam & Łącki. *Fast and Simple Connectivity in Graph Timelines*. *Proceedings of the 14th International Symposium on Algorithms and Data Structures (WADS 2015)*, τόμος 9214 στο *Lecture Notes in Computer Science*, σελίδες 458–469. Springer, Cham, 2015. <https://doi.org/10.1145/502090.502095>.
- [14] Jacob Holm, Eva Rotenberg και Christian Wulff-Nilsen. *Faster Fully-Dynamic Minimum Spanning Forest*. *Proceedings of the 23rd Annual European Symposium on Algorithms (ESA 2015)* Nikhil Bansal και Irene Finocchi, επιμελητές, τόμος 9294 στο *Lecture Notes in Computer Science*, σελίδες 742–753. Springer, Heidelberg, 2015. https://doi.org/10.1007/978-3-662-48350-3_62.
- [15] Shang En Huang, Dawei Huang, Tsvi Kopelowitz και Seth Pettie. *Fully Dynamic Connectivity in $O(\log n (\log \log n)^2)$ Amortized Expected Time*, 2017. <https://epubs.siam.org/doi/10.1137/1.9781611974782.32>.
- [16] Jacob Holm, Wojciech Nadara, Eva Rotenberg και Marek Sokołowski. *Fully dynamic biconnectivity in $\tilde{O}(\log^2 n)$ time*, 2025. <https://arxiv.org/abs/2503.21733>.
- [17] Douglas B. West. *Introduction to Graph Theory*. Prentice Hall, 2η έκδοση, 2001.
- [18] Thomas H. Cormen, Charles E. Leiserson και Ronald L. Rivest & Clifford Stein. *Introduction to Algorithms*. MIT Press, 2009.
- [19] Joakim Skårding, Bogdan Gabrys και Katarzyna Musiał. *Foundations and Modelling of Dynamic Networks using Dynamic Graph Neural Networks: A Survey*. *arXiv preprint arXiv:2005.07496*, 2020. <https://arxiv.org/abs/2005.07496>.
- [20] Farzad V. Farahani, Waldemar Karwowski και Nichole R. Lighthall. *Application of Graph Theory for Identifying Connectivity Patterns in Human Brain Networks: A Systematic Review*. *Frontiers in Neuroscience*, 13:585, 2019. <https://www.frontiersin.org/articles/10.3389/fnins.2019.00585/full>.
- [21] Xuan Zhanget al. *Social Network Analysis and Churn Prediction in Telecommunications Using Graph Theory*. *Entropy*, 22(7):753, 2020. <https://www.mdpi.com/1099-4300/22/7/753>.
- [22] Camil Demetrescu, David Eppstein, Zvi Galil και Giuseppe F. Italiano. *Dynamic Graph Algorithms*. *Algorithms and Theory of Computation Handbook: General Concepts and Techniques* M. J. Atallah και Marina Blanton, επιμελητές, κεφάλαιο 9, σελίδες 9–1–9–28. CRC Press, 2η έκδοση, 2009. <https://dl.acm.org/doi/pdf/10.5555/1882757.1882766>.

- [23] Mikkel Thorup. *Decremental Dynamic Connectivity*. *Journal of Algorithms*, 33(2):229–243, 1999. <https://doi.org/10.1006/jagm.1999.1033>.
- [24] Christian Wulff-Nilsen. *Fully-Dynamic Minimum Spanning Forest with Improved Worst-Case Update Time*. *Proceedings of the 49th Annual ACM SIGACT Symposium on Theory of Computing (STOC 2017)*, σελίδες 1130–1143. ACM, 2017. <https://doi.org/10.1145/3055399.3055415>, or <https://arxiv.org/abs/1611.02864>.
- [25] Frank K. Boesch και S. Chen. *A Generalization of Line Connectivity and Optimally Invulnerable Graphs*. *SIAM Journal on Applied Mathematics*, 34(4):657–665, 1978. <https://epubs.siam.org/doi/10.1137/0134052>, or <https://www.jstor.org/stable/2100729>.
- [26] Wikipedia. *K-edge-connected Graph*, 2025. https://en.wikipedia.org/wiki/K-edge-connected_graph.
- [27] GeeksforGeeks. *Find Bridges in a Graph Using Tarjan’s Algorithm*. <https://www.geeksforgeeks.org/bridge-in-a-graph/>.
- [28] R. E. Tarjan. *Depth-First Search and Linear Graph Algorithms*. *SIAM Journal on Computing*, 1(2):146–159, 1972. <https://doi.org/10.1137/0201010>.
- [29] Codeforces. *Bridge Tree Algorithm - Implementation in C++*. <https://codeforces.com/blog/entry/99259>.

Συντομογραφίες - Αρκτικόλεξα - Ακρωνύμια

Παν. Ιωαννίνων	Πανεπιστήμιο Ιωαννίνων
Ε.Μ.Π.	Εθνικό Μετσόβιο Πολυτεχνείο
βλ.	βλέπε
Κεφ.	Κεφάλαιο
DFS	Deep-First-Search (αναζήτηση κατά βάθος)
P.S.S.	(Richard) Peng, (Bryce) Sandlund & (Daniel D.) Sleator
S.R.A.	Simple Reference Algorithm

Απόδοση ξενόγλωσσων όρων

γράφημα	graph
ακατεύθυντο/ μη-κατευθυνόμενο	undirected
αραιοποίηση	sparsification
σύνδεση / συνεκτικότητα / συνδεσιμότητα	connectivity
δυναμικός	dynamic
ακμή	edge
γέφυρα	bridge
συνεκτικότητα ακμών	edge connectivity
δισυνεκτικότητα ακμών	2-edge connectivity
τρिसυνεκτικότητα ακμών	3-edge connectivity
μόνιμες ακμές	permanent edges
x-ακμική συνεκτικότητα	x-edge connectivity
x-ακμικά συνδεδεμένα	x-edge connected
κόμβος	node
κορυφή	vertex
άρθρωση	articulation point
συνεκτικότητα κορυφών	vertex connectivity
δισυνεκτικότητα κορυφών	2-vertex connectivity
τρिसυνεκτικότητα κορυφών	3-vertex connectivity
ενεργές κορυφές	active vertices
x-κομβική συνεκτικότητα	x-vertex connectivity
x-κομβικά συνδεδεμένα	x-vertex connected
αντιπρόσωπος	representative
χάρτης κατακερματισμού	hash map
μοντέλο	framework
διαίρει και βασίλευε	divide and conquer
αλγόριθμος	algorithm
χρονική πολυπλοκότητα	time complexity
αποσβεστικός	amortized
λειτουργίες	operations
ακολουθία	sequence
ερώτημα	query
προεπεξεργασία	preprocessing
βέλτιστο	optimal
υλοποίηση	implementation

πλήθος	number
εισαγωγή	insertion
διαγραφή	deletion
πλήρως	fully
μερικώς	partially

